An Adaptive First Order Bundle Method for Computing the Linear Scalarization Front of Smooth Nonconvex Multi-objective Optimization

Jiahui (Jeffrey) Cheng
University of California, Berkeley
Berkeley, CA 94720
jeffrey_cheng@berkeley.edu

Rongchuan Shi
University of California, Berkeley
Berkeley, CA 94720
rongchuan@berkeley.edu

Hua Ye
University of California, Berkeley
Berkeley, CA 94720
hua01@berkeley.edu

Cheng Zeng
University of California, Berkeley
Berkeley, CA 94720
cheng_zeng@berkeley.edu

Paul Grigas
University of California, Berkeley
Berkeley, CA 94720
pgrigas@berkeley.edu

Abstract

Multi-objective optimization (MOO) requires balancing multiple potentially conflicting criteria. It finds applications where the goal is to generate solutions that represent diverse user preferences. A traditional practice to solve MOO is linear scalarization (LS) via a weight vector. However, in reality, the right weights are rarely known at prior to the decision maker. Existing approaches solve the MOO problem at one single weight. We propose an adaptive bundle method that systematically reuses first-order oracle information of the objective functions, in order to generate ϵ-linear scalarization front for all weight vectors. Our method scales up to a general number of K objective functions. Theoretically, our method achieves a better complexity result than the uniform discretization method in terms of first-order oracle calls. Empirically, we demonstrate the advantage of our bundle method over the uniform discretization method on structured problems, including multi-class classification, multi-objective gymnasium, and multi-objective LLM alignment. The code is available at `https://github.com/MarshmallowJeffrey/First-order-method-for-smooth-MOO`.

1 Introduction

Many problems in machine learning involve the simultaneous optimization of multiple objective functions. In physics-informed neural networks (PINNs) training, multiple loss terms induced by PDE residuals and boundary or initial conditions must be optimized simultaneously [YBHNL26]. In multi-task learning, a shared model must perform well across several tasks at once, each with its own loss function [R97]. In multi-class classification, the per-class losses define a collection of objectives that must be jointly minimized to achieve good predictive performance across all classes [Bis06]. These settings—and others such as hyperparameter optimization, federated learning, and fairness-aware training—give rise to the *multi-objective optimization* (MOO) problem.

Preprint.

Formally, let θ denote the decision variable in the Euclidean space $\mathbb{R}^d$, MOO seeks to minimize K objectives $F_k : \mathbb{R}^d \to \mathbb{R}$:

$$\min_{\theta \in \mathbb{R}^d} \{F(\theta) := [F_1(\theta), \ldots, F_K(\theta)]^T\}.$$

A common approach to solve MOO is via *linear scalarization*. For example, in the bi-objective case ($K = 2$), given $\lambda \in \Delta_2$, linear scalarization (LS) solves

$$\theta^*_\lambda \in \arg\min_{\theta \in \mathbb{R}^d} \{F_\lambda(\theta) := \lambda_1 F_1(\theta) + \lambda_2 F_2(\theta)\}.$$

For a single, fixed λ, this reduces to a scalar optimization problem. However, for many machine learning and operations research applications, the weights need to be assigned at prior. One potential resolution is to find good solutions for *all* weight vectors $\lambda \in \Delta_2$—that is, to approximate the entire linear scalarization front. The standard approach is uniform discretization, which partitions Δ_2 into a uniform grid and solves the multi-objective optimization problem at each grid point. Solutions to the MOO problem at other grid points are obtained by rounding to the closest grid point. However, as K scales up, the oracle complexity also scales up exponentially, which makes the uniform discretization approach intractable. This motivates the question:

Is there an **efficient** alternative to solve the MOO problem **for all** weight vectors in the unit simplex?

We answer this by proposing a *first-order bundle method*. The key idea is to maintain a *bundle*—a collection of first-order oracle information, consisting of function and gradient value of the objective functions—and to construct from this bundle a *progress criterion* that certifies solution quality *uniformly* over all grid points in the unit simplex.

1.1 Our Contributions

- **C1) A bundle algorithm with convergence guarantee.** In Section 3, we propose a first-order bundle method (Algorithm 1), which adaptively selects a weight vector λ and solves the MOO problem for all weight vectors in the unit simplex for a general number of K objective functions. We establish oracle complexity bounds for different choices of inner solvers inside Algorithm 1.
- **C2) Empirical verification.** In section 4, we present numerical results on diverse tasks, including multi-class classification, MO-Gymnasium benchmarks, and LLM alignment tasks. Our bundle method outperforms the uniform discretization method for large K under fixed total computation budgets.

1.2 Related Work

There has been very little work that addresses this challenge. Related methods such as the Multiple Gradient Descent Algorithm (MGDA) [Dés12] and its variants [SK18, LLJ$^+$21, NSA$^+$22] seek a single Pareto-stationary point by combining task gradients, but they do not characterize the full linear scalarization front. Grid-based scalarization methods [SK18] discretize Δ_K and solve each subproblem independently, but this is wasteful: gradient information computed at one weight vector is discarded when solving for another. Neither approach provides a principled mechanism for *reusing* first-order information across different scalarization weights. In addition, most works [ZLS$^+$24, JHC26] solve this problem for $K = 2$, and there has been little work that addresses this challenge for a general number of objective functions.

2 Model Setup and Preliminaries

In this work, we consider the following continuous multi-objective optimization (MOO) problem [LZY+24] with K differentiable objective functions $F = \{F_1, \ldots, F_K\} : \prod_{i=1}^{K} \mathbb{R}^d \to \mathbb{R}^K$.

$$\min_{\theta \in \mathbb{R}^d} F(\theta) = (F_1(\theta), \ldots, F_K(\theta)). \tag{1}$$

For a non-trivial problem 1, the objective functions $\{F_1, \ldots, F_K\}$ will conflict with each other and cannot be simultaneously optimized by a single best solution.

Notation. For any positive integer m, let $[m]$ denote the set $\{1, \ldots, m\}$. Let $\Delta_K := \{\lambda \in \mathbb{R}^K : \sum_{k=1}^{K} \lambda_k = 1, \lambda \geq 0\}$ be the unit simplex of dimension $K-1$. Let $\nabla F_k(\theta)$ be the gradient of function $F_k : \mathbb{R}^d \to \mathbb{R}$ at point $\theta \in \mathbb{R}^d$. We will use $\|\cdot\|_2$ on $\mathbb{R}^d$ and $\|\cdot\|_1$ on Δ_K.

We first impose the following assumptions on each objective function F_k, and discuss the linear scalarization approach to solve multi-objective optimization (MOO) problem 1.

Assumption 2.1 (Smoothness)**.** *Assume each $F_k : \mathbb{R}^d \to \mathbb{R}$ is L_k-smooth with respect to the l_2-norm, i.e: $\|\nabla F_k(\theta_1) - \nabla F_k(\theta_2)\|_2 \leq L_k \|\theta_1 - \theta_2\|_2, \forall \theta_1, \theta_2 \in \mathbb{R}^d$.*

Assumption 2.2 (Boundedness)**.** *Assume each $F_k : \mathbb{R}^d \to \mathbb{R}$ is bounded below, i.e: $F_k^* := \inf_{\theta \in \mathbb{R}^d} F_k(\theta) > -\infty$. Note that finiteness of F_k^* is not a strong assumption in many applications that we care about; for example, in machine learning most loss functions are nonnegative.*

One classical and popular method for tackling multi-objective optimization is the simple linear scalarization [Geo67] approach:

$$\theta_\lambda^* \in \arg\min_{\theta \in \mathbb{R}^d} \{F_\lambda(\theta) := \sum_{k=1}^{K} \lambda_k F_k(\theta)\}, \tag{2}$$

where $\lambda \in \Delta_K$ is the preference vector on the simplex Δ_K.

Remark 2.1. *Note that $F_\lambda(\cdot)$ is L_λ-smooth with $L_\lambda := \sum_{k=1}^{K} \lambda_k L_k$, bounded below with $F_\lambda^* := \inf_{\theta \in \mathbb{R}^d} F_\lambda(\theta) \geq \sum_{k=1}^{K} \lambda_k F_k^* > -\infty$, and non-convex.*

Once a preference λ is given, we can obtain a solution by solving the single-objective scalarization problem 2. However, in reality, decision makers usually don't know their preferences at prior. Hence, an alternative approach is to recover the entire linear scalarization front

$$\{F_\lambda^* : \lambda \in \Delta_K\} = \{F_\lambda(\theta^*(\lambda)) : \lambda \in \Delta_K\}, \tag{3}$$

i.e: solve the optimization problem 2 for all weight vector $\lambda \in \Delta_K$.

We use an *approximate solution map* $\hat{\theta}(\cdot) : \Delta_K \to \mathbb{R}^d$ to track the set of solutions. To evaluate the quality of an approximate solution map in the non-convex setting, we use the following measure of worst-case solution map stationarity:

$$\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) := \max_{\lambda \in \Delta_K} \|\nabla F_\lambda(\hat{\theta}(\lambda))\|. \tag{4}$$

Note that

$$\{F_\lambda(\theta^*(\lambda)) : \lambda \in \Delta_K\} = \{F_\lambda(\hat{\theta}(\lambda)) : \nabla F_\lambda(\hat{\theta}(\lambda)) = 0, \lambda \in \Delta_K\} = \{F_\lambda(\hat{\theta}(\lambda)) : \max_{\lambda \in \Delta_K} \|\nabla F_\lambda(\hat{\theta}(\lambda))\| = 0\}.$$

Hence, our goal is to develop an adaptive bundle algorithm, with corresponding oracle complexity analysis, to find an approximate solution map $\hat{\theta}(\cdot)$ satisfying $\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) \leq \epsilon$ for a given error tolerance $\epsilon > 0$.

We consider two types of first-order oracles: a deterministic first-order oracle and a stochastic first-order oracle.

Definition 2.1 (Deterministic first-order oracle)**.** *A deterministic first-order oracle for a function $F_k : \mathbb{R}^d \to \mathbb{R}$ takes as input a query point $\bar{\theta} \in \mathbb{R}^d$ and returns a deterministic first-order pair $(F_k(\bar{\theta}), \nabla F_k(\bar{\theta}))$.*

Definition 2.2 (Stochastic first-order oracle)**.** *A stochastic first-order oracle for a finite-sum function* $F_k(\theta) = \frac{1}{n}\sum_{j=1}^{n} F_k(\theta, z_j)$ *takes as input a query point* $\bar{\theta} \in \mathbb{R}^d$*, a data point* z_j*, and returns a stochastic first-order pair* $(F_k(\bar{\theta}, z_j), G(\bar{\theta}, z_j))$ *where* $G(\bar{\theta}, z_j)$ *is the gradient of* $F_k(\theta, z_j)$*.*

Definition 2.3 (Bundle of deterministic first-order oracles)**.** *A bundle of deterministic first-order oracles for functions* $\{F_1, \ldots, F_k\}$*, computed at a list of points* $\theta_1, \ldots, \theta_t$*, is denoted by*

$$\mathcal{B}_t := \{(\theta_i, F_1(\theta_i), \ldots, F_K(\theta_i), \nabla F_1(\theta_i), \ldots, \nabla F_K(\theta_i))\}_{i=1}^{t}.$$

Definition 2.4 (Gradient norm function)**.** *A "gradient norm" function* $\mathrm{GN}(\cdot; \mathcal{B}_t) : \Delta_K \to \mathbb{R}$ *associated with bundle* $\mathcal{B}_t$ *is denoted by*

$$\mathrm{GN}(\lambda; \mathcal{B}_t) := \min_{\theta_i \in \mathcal{B}_t} \{\|\nabla F_\lambda(\theta_i)\|\}.$$

Proposition 1 (Global Monotonicity of $\mathrm{GN}(\lambda; \cdot)$)**.** *For any bundles* $\mathcal{B} \subseteq \mathcal{B}'$*,*

$$\mathrm{GN}(\lambda; \mathcal{B}') \leq \mathrm{GN}(\lambda; \mathcal{B}), \text{ for all } \lambda \in \Delta_K.$$

3 An adaptive bundle algorithm for computing the linear scalarization front

The most common approach to solve the MOO problem (2) is the uniform discretization method, which partitions the unit simplex Δ_K, solve the optimization problem defined at each grid point, and build the solution map. For a detailed description, see appendix B.1. However, this approach can be wasteful, as the MOO problem defined at each grid point is solved independently, with discarded first-order oracle information that could be useful for solving MOO problems at other grid points.

With this observation, we propose an adaptive bundle algorithm for computing the linear scalarization front of the MOO problem (2), which is both theoretically and practically more efficient than the uniform discretization method in terms of first-order oracle complexity. At a high level, the adaptive algorithm builds a bundle of deterministic first-order oracles for the objective function, as well as an associated progress criterion (4) characterizing the worst-case solution quality over the unit simplex.

Algorithm 1 Adaptive bundle algorithm

Input: Max outer iterations T; initial point θ_0; error tolerance ϵ; bundle point select rule $\mathcal{R}$.
Initialize: Initial bundle $\mathcal{B}_0$ built at θ_0.
for iteration $t = 1, \ldots, T$ **do**
  Compute $\lambda_t \in \Delta_K$ maximizing $\min_{\theta_i \in \mathcal{B}_{t-1}} \|\nabla F_{\lambda_t}(\theta_i)\|$; set $\mathrm{GN}_t^* := \mathrm{GN}(\lambda_t; \mathcal{B}_{t-1})$ to be the maximum value.
  if $\mathrm{GN}_t^* \leq \epsilon$ **then**
    return bundle $\mathcal{B}_{t-1}$ to build solution map $\hat{\theta}(\lambda) := \arg\min_{\theta_i \in \mathcal{B}_{t-1}} \|\nabla F_\lambda(\theta_i)\|$.
  else
    CandidatePoints = InnerSolver $(\lambda_t; \mathcal{B}_{t-1})$
    $\mathcal{B}_t = \mathcal{B}_{t-1} \cup \mathcal{R}$(CandidatePoints)
  end if
end for

Preference weight selection. At outer iteration t, we select a preference weight from the unit simplex by solving the following optimization problem:

$$\mathrm{GN}(\lambda_t; \mathcal{B}_{t-1}) := \max_{\lambda_t \in \Delta_K} \min_{\theta_i \in \mathcal{B}_{t-1}} \|\nabla F_{\lambda_t}(\theta_i)\|. \tag{5}$$

Note that the objective function is a maximum of a pointwise minimum of convex functions, which is non-concave and NP-hard in general.

To solve this optimization problem, we propose a multi-start convex-concave procedure (CCP). The idea is to generate a sequence of non-decreasing lower bounds $\{\phi(\lambda_t^c)\}_{c \geq 0}$ of the objective function that converges to stationary points of the optimization problem. The generation of each such lower bound $\phi(\lambda_t^c)$ reduces to solving a linear program. Theoretically, once the CCP finds a preference weight λ_t^c such that $\mathrm{GN}(\lambda_t^c, \mathcal{B}_{t-1}) > \epsilon$, the procedure can be terminated, and the "else" clause will be invoked. This observation could potentially save computational cost required for generating the sequence of lower bounds. But in practice, it might be more intuitive to search for the preference weight that attains the worst-case optimum, as it is more informative as a bundle point for later preference weight selection. For implementation details, see Appendix B.2.

One could argue that the computational cost to solve the preference weight selection problem can be expensive, which is true as the cost of forming the Gram matrices grows with the bundle size $|\mathcal{B}_t|$ and the number of objectives K, and a LP is required to be solved per CCP iteration per seed. However, we emphasize here that we are only interested in settings where the gradient oracle call is expensive, with computation cost outweighs the CCP computation cost.

Solution map construction. To build solution map $\hat{\theta}(\lambda) := \arg\min_{\theta_i \in \mathcal{B}_t} \|\nabla F_\lambda(\theta_i)\|$, the least-index rule is used to break ties. Note that the solution map $\hat{\theta}(\cdot)$ is cheap to query, as it reduces to $\|\mathcal{B}_t\|$ evaluations of $\|\nabla F_\lambda(\theta_i)\|^2 = \lambda^T Q(\theta_i) \lambda$ with precomputed Gram matrices $Q(\theta_i)$ for $\theta_i \in \mathcal{B}_t$.

Inner Solver. The purpose of the inner solver is to generate a sequence of candidate points that achieve a prescribed gradient-norm accuracy with prescribed probability. See Assumption 3.1. It also implements the warm-start strategy that takes the best point from the last iterate bundle as the starting point for the current iterate, which could improve the computational efficiency. Many

first-order algorithms can be employed to generate this sequence of candidate points. In this paper, we analyze five different methods: vanilla gradient descent (GD), randomized stochastic gradient descent (RSGD), a two-phase randomized stochastic gradient descent (2-RSGD), stochastic variance reduced gradient method (SVRG), and a two-phase stochastic variance reduced gradient method (2-SVRG).

Bundle point select rule. After the inner solver generates a list of candidate points, we may update the current bundle by selecting a subset of them. There are many selection rules, e.g: adding only the last point, adding all points, adding a random point, or adding the point with the minimum gradient norm. There are trade-offs among these selection rules. For example, adding all points could improve solution accuracy at the cost of extra oracle calls.

We make the following assumptions about the inner solver and bundle point select rule.

Assumption 3.1 (InnerSolver and bundle point select rule)**.** *For every $\eta > 0$ and $\delta \in (0, 1)$, there exists $M(\eta, \delta)$ such that at any outer iteration t, running the inner solver at preference weight λ_t for $M(\eta, \delta)$ iterations and the updated bundle $\mathcal{B}_t$ satisfies the gradient norm precision requirement, i.e:*

$$\mathrm{GN}(\lambda_t; \mathcal{B}_t) \leq \eta, \text{ with probability at least } 1 - \delta.$$

Convergence analysis. Due to potential stochasticity of bundle point trajectory $\{\theta_i\}_{i=1}^m$ generated by InnerSolver, we analyze the convergence rate of Algorithm 1 in terms of probability bound.

We may sometimes need the following bounded algorithm trajectory assumption.

Assumption 3.2 (Bounded Algorithm Trajectory)**.** *There exists a compact set $\Theta \subseteq \mathbb{R}^d$ such that any bundle point trajectory generated by Algorithm 1 remains in Θ.*

Proposition 2. *If InnerSolver is defined using the deterministic gradient descent method, and each objective function F_k satisfies the bounded level set assumption, i.e: the sublevel sets of F_k are bounded, then Assumption 3.2 is satisfied.*

Proposition 3. *Under Assumption 3.2, there exists a constant $\mathrm{LipGN} > 0$, possibly depending on the size of Θ, such that the functions $\mathrm{GN}(\cdot; \mathcal{B})$ for all bundles $\mathcal{B}$ restricted to Θ are uniformly Lipschitz, i.e., for any bundle $\mathcal{B}$ restricted to Θ we have*

$$|\mathrm{GN}(\lambda_1; \mathcal{B}) - \mathrm{GN}(\lambda_2; \mathcal{B})| \leq \mathrm{LipGN} \|\lambda_1 - \lambda_2\|_1 \text{ for all } \lambda_1, \lambda_2 \in \Delta_K.$$

Theorem 1 (Convergence of Adaptive Bundle Algorithm 1)**.** *Under Assumptions 2.1, 2.2, 3.1 and 3.2, given error tolerance $\epsilon > 0$ and inner solver failure rate $\delta \in (0, 1)$, Algorithm 1 terminates with $\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) \leq \epsilon$, and the number of outer iterations T satisfies $Y \succeq_{st} T$ for $Y \sim$ Pascal$(\lceil T_C(\epsilon, K) \rceil, 1 - \delta)$ where $T_C(\epsilon, K) = \left(1 + \frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1}$ for some constant $C > 0$; in particular $\mathbb{E}[T] \leq \frac{\lceil T_C(\epsilon,K) \rceil}{1-\delta}$.*

We could compare the convergence rate of the Adaptive Bundle Algorithm 1 with the Uniform Discretization Algorithm 7 as the baseline method. For details, see Appendix B.1.

Proposition 4 (Convergence of Uniform Discretization Algorithm 7)**.** *Under Assumptions 2.1, 2.2, 3.2 and 3.1, given error tolerance $\epsilon > 0$, Algorithm 7 terminates with $\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) \leq \epsilon$, and the number of grid points T is upper bounded by $e^K \left(1 + \frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1}$ for some constant $C > 0$.*

3.1 Convergence results of inner solvers

We present five inner solvers: vanilla gradient descent (GD) inner solver, randomized stochastic gradient descent (RSGD) inner solver, a two-phase randomized stochastic gradient descent (2-RSGD) inner solver, stochastic variance reduced gradient (SVRG) inner solver, and a two-phase stochastic variance reduced gradient (2-SVRG) inner solver.

Algorithm 2 InnerSolver (GD)

Input: iteration limit M; bundle $\mathcal{B}_{t-1}$; error tolerance ϵ; step sizes $\{\gamma_m\}_{m=1}^M$
Initialize: $\theta_0 = \arg\min_{\theta \in \mathcal{B}_{t-1}} \{F_\lambda(\theta) - \frac{1}{2L_\lambda}\|\nabla F_\lambda(\theta)\|^2\}$; CandidatePoints $= \{\theta_0\}$.
for iteration $m = 1, \ldots, M$ **do**
  $\theta_m = \theta_{m-1} - \gamma_m \nabla F_\lambda(\theta_{m-1})$
  CandidatePoints $=$ CandidatePoints $\cup \{\theta_m\}$
end for
return CandidatePoints

Algorithm 3 InnerSolver (RSGD)

Input: Iteration limit M; bundle $\mathcal{B}_{t-1}$; error tolerance ϵ; step sizes $\{\gamma_m\}_{m=1}^M$; probability mass function $P_R(\cdot)$; data $D := \{z_1, \ldots, z_n\}$
Initialize: $\theta_0 = \arg\min_{\theta \in \mathcal{B}_{t-1}} \{F_\lambda(\theta) - \frac{1}{2L_\lambda}\|\nabla F_\lambda(\theta)\|^2\}$; R is a random variable with probability mass function P_R.
for iteration $m = 1, \ldots, R$ **do**
  Uniformly randomly sample z^m from D
  $\theta_m = \theta_{m-1} - \gamma_m G_\lambda(\theta_{m-1}, z^m)$
end for
return $\{\theta_R\}$

Algorithm 4 InnerSolver (2-RSGD)

Input: Iteration limit M; bundle $\mathcal{B}_{t-1}$; error tolerance ϵ; step sizes: $\{\gamma_m\}_{m=1}^M$; probability mass function $P_R(\cdot)$; data $D := \{z_1, \ldots, z_n\}$; Number of runs S
CandidatePoints = Amplify(RSGDInnerSolver($M, \mathcal{B}_{t-1}, \epsilon, \{\gamma_m\}_{m=1}^M, P_R, D$), S)
return CandidatePoints

Algorithm 5 InnerSolver (SVRG)

Input: Iteration limit M; bundle $\mathcal{B}_{t-1}$; error tolerance ϵ; step sizes $\{\gamma_m\}_{m=1}^M$; data $D := \{z_1, \ldots, z_n\}$; Number of epochs Q
Initialize: $\theta^0 = \theta_M^0 = \tilde{\theta}^0 = \arg\min_{\theta \in \mathcal{B}_{t-1}} \{F_\lambda(\theta) - \frac{1}{2L_\lambda}\|\nabla F_\lambda(\theta)\|^2\}$; CandidaPoints $= \{\theta^0\}$.
for epoch $q = 0, \ldots, Q-1$ **do**
  $\theta_0^{q+1} = \theta_M^q$
  $g^{q+1} = \frac{1}{n}\sum_{j=1}^n G_\lambda(\tilde{\theta}^q, z_j)$
  for iteration $m = 0, \ldots, M-1$ **do**
    Uniformly randomly pick z^m from D
    $v_m^{q+1} = G_\lambda(\theta_m^{q+1}, z^m) - G_\lambda(\tilde{\theta}^q, z^m) + g^{q+1}$
    $\theta_{m+1}^{q+1} = \theta_m^{q+1} - \gamma_m v_m^{q+1}$
    CandidatePoints $=$ CandidatePoints $\cup \{\theta_{m+1}^{q+1}\}$
  end for
  $\tilde{\theta}^{q+1} = \theta_M^{q+1}$
end for
return CandidatePoints

Algorithm 6 InnerSolver (2-SVRG)

Input: Iteration limit M; bundle $\mathcal{B}_{t-1}$; error tolerance ϵ; step sizes $\{\gamma_m\}_{m=1}^M$; data $D := \{z_1, \ldots, z_n\}$; number of epochs Q; number of runs S;
CandidatePoints = Amplify(SVRGInnerSolver($M, \mathcal{B}_{t-1}, \epsilon, \{\gamma_m\}_{m=1}^M, D, Q$), S)
return CandidatePoints

We next present the convergence proofs of these inner solvers with various bundle update rules and the corresponding oracle complexity. At first, we need the following corollary.

Corollary 3.1 (Oracle Complexity of Inner Solver)**.** *Let Γ denote the number of oracle calls made after $M(\eta, \delta)$ iterations of inner solver at accuracy η and failure rate δ. Then, $\Gamma = \Gamma_{admit} + \Gamma_{solve}$ splits into the cost of obtaining exact first-order information at the points the select rule retains, and the cost of the inner solver. In the deterministic setting, $\Gamma_{admit} = 0$; in the stochastic setting, $\Gamma_{admit} = RnK$ for R retained points. In both cases, $\Gamma_{solve} = KM(\eta, \delta)$.*

We make the following assumptions for stochastic inner solvers under the finite-sum function setting.

Assumption 3.3 (Unbiasness and bounded variance)**.** *There exists a parameter $\sigma > 0$ such that*

$$\mathbb{E}_{x\sim D}\big[G_\lambda(\theta, x)\big] = \nabla F_\lambda(\theta) \quad \textit{and} \quad \mathbb{E}_{x\sim D}[\|G_\lambda(\theta, x) - \nabla F_\lambda(\theta)\|^2] \le \sigma^2, \textit{ for all } \theta \in \mathbb{R}^d, \textit{for all } \lambda \in \Delta_K.$$

Assumption 3.4 (Per-component smoothness of the finite-sum function)**.** *For finite-sum function $F_k(\theta) = \frac{1}{n}\sum_{j=1}^n F_k(\theta, z_j)$, we assume $F_k(\theta, z_j)$ is L_k-smooth for each data vector z_j.*

In order to establish the convergence proofs of the two-phase stochastic gradient descent (2-SGD) inner solver and the two-phase stochastic variance reduced gradient descent (2-SVRG) inner solver, we need the following lemma.

Lemma 1 (Amplification)**.** *For some constant $\alpha \in (0, 1]$ and $C > 0$, suppose a stochastic base solver returns, after M iterations, a candidate bundle point $\bar{\theta}$ at preference weight λ such that*

$$\mathrm{GN}(\lambda; \{\bar{\theta}\}) \le \frac{C}{M^\alpha}, \textit{ with probability at least } \frac{1}{2}.$$

Then, running S independent copies and retaining the bundle $\mathcal{B}$ of candidate points gives

$$\mathrm{GN}(\lambda; \mathcal{B}) \le \frac{C}{M^\alpha}, \textit{ with probability at least } 1 - \frac{1}{2^S}.$$

Hence, $S := \lceil \log_2 \frac{1}{\delta} \rceil$ suffices for failure rate δ, at admission cost $\Gamma_{admit} = SnK$.

Proposition 5 (Convergence of inner solvers paired with various bundle point select rules)**.** *Under Assumptions 2.1 and 2.2, given preference weight λ and inner solver failure rate $\delta \in (0, 1)$, we define the following constants and impose the following design choices for these inner solvers and bundle point select rules.*

Inner solver	Bundle update rule	Assumption	Step sizes γ_m	$C_\lambda(n, \delta)$	α	Extra constants
GD	full iterate	—	$\frac{1}{L_\lambda}$	$\sqrt{2L_\lambda[F_\lambda(\theta_0) - F_\lambda^*]}$	$\frac{1}{2}$	—
RSGD	random sampling	Assump 3.3	$\min\{\frac{1}{L_\lambda}, \frac{\tilde{D}_\lambda}{\sigma\sqrt{M}}\}$	$\sqrt{\frac{L_\lambda(L_\lambda\tilde{D}_\lambda^2 + 2\tilde{D}_\lambda\sigma)}{\delta}}$	$\frac{1}{4}$	$\tilde{D}_\lambda := \sqrt{\frac{2[F_\lambda(\theta_0) - F_\lambda^*]}{L_\lambda}}$
2-RSGD	min gradient norm	Assump 3.3	$\min\{\frac{1}{L_\lambda}, \frac{\tilde{D}_\lambda}{\sigma\sqrt{M}}\}$	$\sqrt{L_\lambda(L_\lambda\tilde{D}_\lambda^2 + 2\tilde{D}_\lambda\sigma)}$	$\frac{1}{4}$	same as above
SVRG	random sampling	Assump 3.4	$\frac{\mu_1}{L_\lambda n^{\frac{2}{3}}}$	$\sqrt{\frac{n^{\frac{2}{3}}L_\lambda(L_\lambda\tilde{D}_\lambda^2)}{2\nu_1\delta}}$	$\frac{1}{2}$	μ_1, ν_1 in Corollary 3 [RHS+16]
2-SVRG	min gradient norm	Assump 3.4	$\frac{\mu_1}{L_\lambda n^{\frac{2}{3}}}$	$\sqrt{\frac{n^{\frac{2}{3}}L_\lambda(L_\lambda\tilde{D}_\lambda^2)}{\nu_1}}$	$\frac{1}{2}$	same as above

Table 1: Design choices for inner solvers and bundle point select rules.

In addition, define the probability mass function $P_R(\cdot)$ to be

$$P_R(m) := \mathbb{P}(R = m) = \frac{2\gamma_m - L_\lambda\gamma_m^2}{\sum_{m=1}^M 2\gamma_m - L_\lambda\gamma_m^2}. \tag{6}$$

in the randomized setting.

Let $\mathcal{B}$ be the bundle built after running these inner solvers for M iterations with the corresponding bundle point select rules. Then,

$$\mathrm{GN}(\lambda; \mathcal{B}) \le \frac{C_\lambda(n, \delta)}{M^\alpha}, \textit{ with probability at least } 1 - \delta.$$

Correspondingly, given error tolerance $\eta > 0$, the iteration complexity $M(\eta, \delta)$ and the corresponding oracle complexity Γ required to ensure $\mathrm{GN}(\lambda; \mathcal{B}) \leq \eta$ is summarized as follows.

Inner solver	$M(\eta, \delta)$	$\Gamma = \Gamma_{\mathrm{admit}} + \Gamma_{\mathrm{solve}}$
GD	$\mathcal{O}(\eta^{-2})$	KM
RSGD	$\mathcal{O}(\eta^{-4}\delta^{-2})$	$K(n+M)$
2-RSGD	$\mathcal{O}(\eta^{-4})$	$SK(n+M)$
SVRG	$\mathcal{O}(n^{2/3}\eta^{-2}\delta^{-1})$	$K(n+2M)$
2-SVRG	$\mathcal{O}(n^{2/3}\eta^{-2})$	$SK(n+2M)$

Table 2: Iteration and oracle complexity for different inner solver convergence.

This validates Assumption 3.1.

Inner solver comparison. There are trade-offs in using these inner solvers. For example, the gradient descent inner solver requires evaluating the full gradient along the solution trajectory. This becomes a bottleneck when the dataset is large. Instead, one could use the stochastic setting, where we assume a stochastic gradient oracle $G_k(\theta, z)$ of a finite-sum function $F_k(\theta) = \frac{1}{n}\sum_{j=1}^{n} F_k(\theta, z_j)$ is available. In this setting, the stochastic gradient descent (SGD) inner solver requires only one stochastic gradient oracle call per iteration, which could potentially save computational cost. However, the training schedule can become unstable due to high variance. To address this issue, we could use the stochastic variance reduced gradient (SVRG) inner solver that periodically evaluates the full gradient to reduce the variance incurred along the solution trajectory. We could further improve the oracle complexity bound by employing a two-phase procedure: an optimization phase used to generate a list of candidate solutions via a few independent runs of the inner solver, and a post-optimization phase that updates the bundle by selecting a solution from this candidate list. This technique amplifies the inner solver success rate and reduces the oracle complexity by a logarithmic factor.

3.2 Oracle complexity of Algorithm 1 with different inner solvers

Corollary 3.2 (Oracle Complexity of Algorithm 1)**.** *Given error tolerance $\epsilon > 0$ and inner solver failure rate $\delta \in (0, 1)$, let $\Gamma(\eta, \delta)$ be defined as in Corollary 3.1. Under the hypothesis of Theorem 1,*

$$\mathbb{E}[O] \leq \frac{\lceil T_C(\epsilon, K) \rceil}{1-\delta} \Gamma(\frac{\epsilon}{3}, \delta).$$

We summarize the oracle complexity of algorithm 1 with different inner solvers in the following table.

Inner solver	Oracle complexity
GD	$\mathcal{O}\left(\left(1 + \frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1} K\epsilon^{-2}\right)$
RSGD	$\mathcal{O}\left(\left(1 + \frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1} \frac{K}{1-\delta}(n + \epsilon^{-4}\delta^{-2})\right)$
2-RSGD	$\mathcal{O}\left(\left(1 + \frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1} \frac{K}{1-\delta}\log_2 \frac{1}{\delta}(n + \epsilon^{-4})\right)$
SVRG	$\mathcal{O}\left(\left(1 + \frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1} \frac{K}{1-\delta}\frac{1}{\delta} n^{\frac{2}{3}}\epsilon^{-2}\right)$
2-SVRG	$\mathcal{O}\left(\left(1 + \frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1} \frac{K}{1-\delta}\log_2 \frac{1}{\delta} n^{\frac{2}{3}}\epsilon^{-2}\right)$

Table 3: Oracle complexity of Algorithm 1 with different inner solvers.

4 Numerical Experiments

In this section, we empirically verify the effectiveness of our first-order bundle algorithm 1 against the uniform discretization method 7 as the baseline. We present computational results for various tasks, including multi-class classification, MO-Gymnasium, and LLM fine-tuning.

Worst-case error. In both experiments we compute a bundle $\mathcal{B}_t$: for the uniform-discretisation baseline (Algorithm 7), $\mathcal{B}_t$ is the set of *last iterates* obtained by running gradient descent on F_λ at each node λ of the grid of resolution r; for Algorithm 1, $\mathcal{B}_t$ is the set of points the algorithm accumulates, allocated adaptively to the weights where the gradient norm progress criterion is largest.

We then measure first-order *stationarity* of the scalarised objective along the solution map, reporting the worst-case squared gradient norm

$$\epsilon_{\text{sm-stat}}\big(\widehat{\theta}(\cdot)\big) := \max_{\lambda\in\Delta_K} \big\|\nabla F_\lambda\big(\widehat{\theta}(\lambda)\big)\big\| = \max_{\lambda\in\Delta_K} \min_{\theta_i\in\mathcal{B}_t} \|\nabla F_\lambda(\theta_i)\| = \max_{\lambda\in\Delta_K} \mathrm{GN}(\lambda;\mathcal{B}_t) \tag{7}$$

A small value of (7) certifies that $\widehat{\theta}(\lambda)$ is approximately stationary for F_λ *uniformly* over the simplex Δ_K.

Two budget axes. We benchmark each method along two complementary axes:

1. **Gradient evaluations.** We count one gradient evaluation per call to a single $\nabla F_k(\cdot)$, so each scalarised gradient-descent step costs K evaluations. For applications where the gradient is expensive — for instance, large-scale neural-network training where each ∇F_k is a backpropagation pass — this axis directly reflects wall-clock cost.
2. **CPU time.** We count the total CPU time for both methods.

Checkpoint cadence. For each method, we record (gradient evaluations, CPU time, worst-case error) tuples at periodic checkpoints.

Implementation details of Algorithm 1. For Algorithm 1, we make the following implementation choice that materially affect runtime cost:

- **Inner-loop pruning.** For weight λ_t at iteration t, we add only the point of the last iterate to the current bundle $\mathcal{B}_t$.

4.1 Multiclass classification

We now evaluate the method on multi-objective problems built from real data. From MNIST we construct a $K = 3$ instance: each objective is the mean cross-entropy of one digit class (plus a small ridge term), and the three classes share a single small network, so the objectives compete for capacity and conflict arises naturally. We compare the adaptive λ-bundle against uniform simplex grids of three resolutions under a strictly matched budget of gradient evaluations. Performance is measured by the worst-case squared gradient norm GN^* over the whole simplex and by the resulting training and test Pareto fronts (hypervolume, best mean test loss and test error). Section 4.1.1 first selects the digit triple; Section 4.1.2 reports the comparison on the selected instance.

4.1.1 Instance selection by trajectory-level conflict screening

Before comparing the methods at $K = 3$ we must fix the digit triple that defines the problem instance. A good instance should exhibit strong and persistent conflict among *all three* class objectives, so that the Pareto front is genuinely three-way rather than driven by a single antagonistic pair. To rank candidates we adapt the inter-task affinity ("lookahead") measure from multi-task learning [FAZ$^+$21, SZC$^+$20], used in reverse: task grouping seeks tasks that do *not* interfere, whereas we seek the triple that interferes *most*.

For a triple $D = \{a, b, c\}$, let $F_k(\theta)$ be the mean cross-entropy of the network on the training images of class k. We run one neutral training trajectory per triple, using the same segment executor as the

main experiments with fixed weights $\lambda = (1/3, 1/3, 1/3)$, and at checkpoints θ^t we measure

$$Z^t_{i\to j} \;=\; 1 - \frac{F_j\big(\theta^t - \eta_i \nabla F_i(\theta^t)\big)}{F_j(\theta^t)}, \qquad \eta_i = \alpha / L_i, \tag{8}$$

the relative change of objective j after a *virtual* gradient step on objective i, where L_i is a smoothness estimate of F_i and α a dimensionless lookahead scale (values below). A negative $Z^t_{i\to j}$ means that helping class i hurts class j. The probe parameters are discarded immediately, so the real trajectory is unaffected.

The negative parts are aggregated over the T checkpoints:

$$C_{i\to j} = \frac{1}{T}\sum_t \max\big(0, -Z^t_{i\to j}\big), \qquad c_j = \frac{1}{K-1}\sum_{i\neq j} C_{i\to j}, \qquad C_{\mathrm{bal}} = \min_j c_j.$$

Here c_j is the average conflict flowing *into* digit j, and the ranking score C_{bal} is large only when all three digits are under persistent pressure.

All $\binom{10}{3} = 120$ triples are screened under an identical protocol:

- 5,421 training images per class (the minimum class count), so every triple shares $n = 16{,}263$ and the same gradient budget;
- identical initialization and sampling seeds; per-objective smoothness estimates L_i from random probes;
- $\alpha = 0.05$, checkpoints $t \in \{0, 3, 6, 9, 12, 15\}$ over 15 segments; the official test set is never touched.

Table 4: Trajectory-level conflict screening of all 120 digit triples (top five shown). c_j lists the incoming conflict of each digit; $C_{\mathrm{bal}} = \min_j c_j$ is the ranking score.

Rank	Triple	C_{bal}	c_j
1	$\{4,7,9\}$	1.698	3.20 / 1.70 / 1.92
2	$\{3,5,8\}$	1.683	2.40 / 2.04 / 1.68
3	$\{6,7,9\}$	1.659	1.81 / 1.66 / 1.68
4	$\{3,4,5\}$	1.638	2.52 / 1.69 / 1.64
5	$\{2,4,9\}$	1.618	1.99 / 1.97 / 1.62

Table 4 shows the top of the ranking. The triple $\{4,7,9\}$ ranks first ($C_{\mathrm{bal}} = 1.698$) and $\{3,5,8\}$ second (1.683). Their conflict structure differs. In $\{4,7,9\}$ the conflict concentrates on digit 4 (incoming conflict 3.20 versus $1.70/1.92$), closer to a strong pair plus a bystander. In contrast, $\{3,5,8\}$ is the most balanced triple at the top of the ranking ($2.40/2.04/1.68$). We therefore use $\{3,5,8\}$ for the main experiment.

4.1.2 Comparison under a matched budget

On the selected triple $\{3,5,8\}$ the network ends in a softmax layer, so each objective is a logistic-regression-type loss on shared learned features:

$$F_k(\theta) \;=\; \frac{1}{n_k}\sum_{i:\, y_i = k} -\log p_\theta(k \,|\, x_i) \;+\; \frac{\mu}{2}\|\theta\|^2, \qquad \mu = 10^{-4}, \tag{9}$$

where n_k is the class count. Training at a weight vector λ minimizes the scalarization

$$F_\lambda(\theta) \;=\; \sum_{k=1}^{3} \lambda_k F_k(\theta), \qquad \lambda \in \Delta_2, \tag{10}$$

and since $\sum_k \lambda_k = 1$, penalizing each objective is identical to adding a single ridge term to F_λ. The protocol:

- **Data.** 5,421 training images per class ($n = 16{,}263$, balanced), pixels scaled to $[0, 1]$; the test set is all 2,876 official test images of the three digits and is used for evaluation only.

- **Model.** patch(5×5)-64 $\to$ dense-96 $\to$ 3 logits with softplus activations, $d = 8{,}195$ parameters.
- **Methods**
  - Uniform grids $r \in \{10, 20, 30\}$ ($66/231/496$ nodes, visited in snake order).
  - The adaptive λ-bundle: before each decision it solves a CCP subproblem on the accumulated gradient information ($N_0 = 2{,}000$ samples, inner resolution $r = 10$, exponential sampling, pooling enabled).
- **Budget and inner solver.** $B = 40{,}000$ gradient units for every method, stopping when the budget is spent. Each selected λ is trained for $s = 5$ consecutive segments; one segment is a full-batch anchor gradient followed by 16 mini-batch steps of momentum MSVRG (momentum 0.5). Mini-batches hold $1{,}024$ images, stratified by class ($342/341/341$), drawn without replacement; the step size is $0.1/(\lambda^{\mathsf{T}} L \cdot L_{\mathrm{scale}})$ with per-objective smoothness estimates $L = (1.854,\ 1.775,\ 1.581)$ (ridge included).
- **Fairness.** All four methods share the same seeds (initialization 8, sampling 41, smoothness probes 7, CCP 0) and a bit-identical initial point.

Fronts are scored by hypervolume with reference point $z^{\mathrm{ref}} = (\ln 3, \ln 3, \ln 3)$, the loss of uniform guessing:

$$\mathrm{HV}(\mathcal{F}) \;=\; \mathrm{Vol}\big(\{\, y \in \mathbb{R}^3 : \exists f \in \mathcal{F},\ f \preceq y \preceq z^{\mathrm{ref}} \,\}\big), \tag{11}$$

where $\mathcal{F}$ is the set of non-dominated points and $\preceq$ is the componentwise order; larger is better.

On the test side we additionally track, over all snapshots θ_t produced during a run, the best mean test loss and the best mean test error,

$$\mathcal{L}_{\mathrm{test}} \;=\; \min_t \; \frac{1}{3}\sum_{k=1}^{3} \frac{1}{m_k} \sum_{i:\, y_i = k} -\log p_{\theta_t}(k \,|\, x_i), \tag{12}$$

$$\mathcal{E}_{\mathrm{test}} \;=\; \min_t \; \frac{1}{3}\sum_{k=1}^{3} \frac{1}{m_k} \sum_{i:\, y_i = k} \mathbf{1}\{\hat{y}_i(\theta_t) \neq k\}, \qquad \hat{y}_i(\theta_t) = \arg\max_j z_j(x_i; \theta_t), \tag{13}$$

where m_k is the test count of class k.

Results. Figure 1 tracks GN* against spent budget and against wall-clock time. At the full budget the adaptive method reaches 3.4×10^{-4}, a $19.1\times$ lead over the best grid ($r = 20$, 6.6×10^{-3}); at matched intermediate budgets the lead ranges from $8.5\times$ to $21\times$, and the adaptive run already beats every grid's *final* value using a quarter of the budget. The grid family is non-monotone in r ($r = 20$ best, $r = 10$ worst): the coarse grid loses to coverage gaps near the simplex vertices, the fine grid to insufficient training per node, so no single resolution resolves the trade-off. On the time axis, which includes the decision overhead, the lead is $6.8\times/14.4\times/19.5\times$ at $300/700/1{,}500$ s.

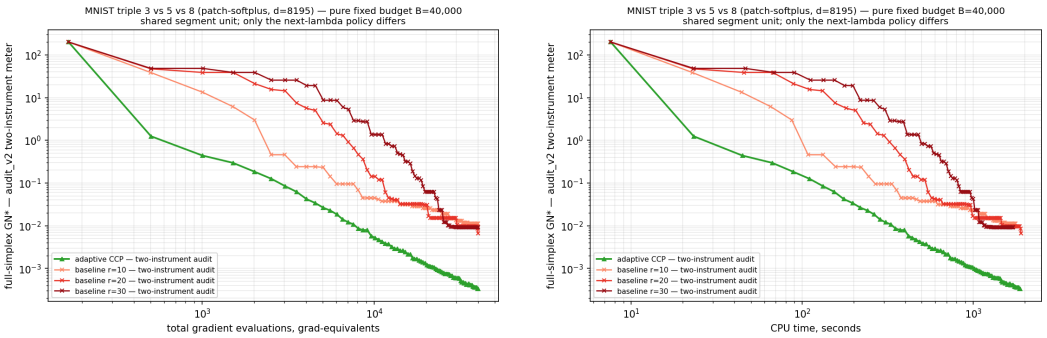

Figure 1: MNIST $\{3, 5, 8\}$: worst-case squared gradient norm versus total gradient evaluations (left) and CPU time (right), at a matched budget of $40{,}000$ gradient units.

Figures 2 and 3 overlay the lower envelopes of the training- and test-set fronts. On the training front the four methods are nearly tied (hypervolume 1.250–1.253), so the GN* advantage does not

come at the cost of front coverage; on the test front the adaptive method is first on all three metrics: hypervolume 1.2989 versus 1.2843 for the best grid, best mean test cross-entropy 0.0483 versus 0.0560, and best mean test error 1.27% versus 1.58%.

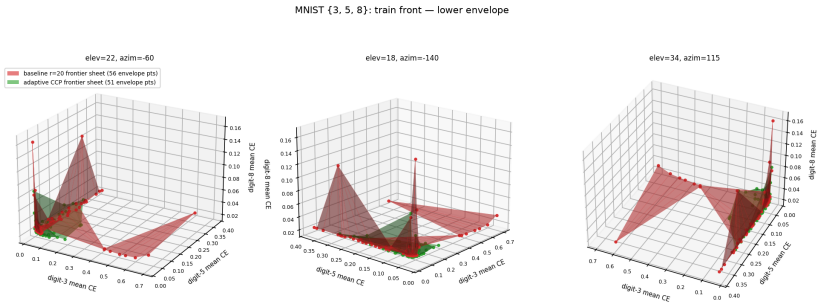

Figure 2: MNIST $\{3, 5, 8\}$: lower envelopes of the training-set fronts from three viewing angles. The four methods are nearly tied (hypervolume 1.250–1.253).

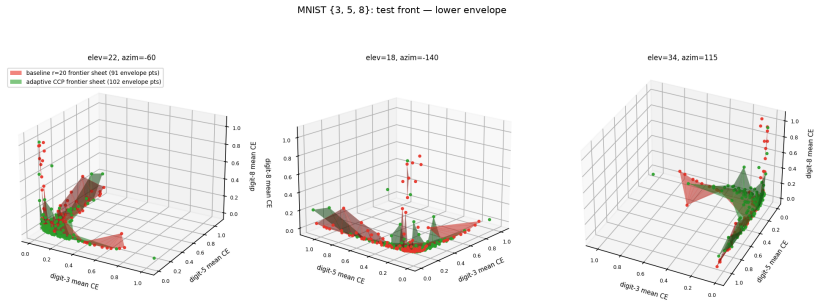

Figure 3: MNIST $\{3, 5, 8\}$: lower envelopes of the test-set fronts from three viewing angles. The adaptive front (green) dominates the best grid ($r = 20$, red) in the low-loss corner; hypervolume 1.2989 versus 1.2843.

4.2 MO-Gymnasium Benchmarks

We now turn to multi-objective reinforcement learning, using three tasks from the MO-Gymnasium benchmark suite: FishWood and Deep Sea Treasure with $K = 2$ objectives, and Breakable Bottles with $K = 3$. All three are finite MDPs, which lets us evaluate each objective and its gradient in closed form, so that the comparison between the two methods is not confounded by gradient noise.

Each task is a regularized multi-objective MDP $\mathcal{M} := (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma, \rho, \tau)$ with a vector-valued reward $r = [r_1, \dots, r_K]$. We parametrize the policy by logits $\theta \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$, so that $\pi_\theta(a \mid s) = \mathrm{softmax}(\theta_{s,\cdot})_a$, and take the k-th objective to be the KL-regularized negative return

$$F_k(\theta) := \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t \Big(\tau\,\mathrm{KL}\big(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid s_t)\big) - r_k(s_t, a_t)\Big)\right], \qquad k = 1, \dots, K, \tag{14}$$

where π_{ref} is the uniform policy. Minimizing F_k maximizes the k-th discounted return, while the KL term keeps the policy away from the boundary of the simplex and makes each F_k smooth on the region the algorithms explore.

Since the state and action spaces are finite, we assemble the transition tensor $\mathcal{P} \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|\times|\mathcal{S}|}$ and the reward tensor $r \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|\times K}$ explicitly. The discounted state-occupancy measure is then available in closed form,

$$d_{\pi_\theta} = (1-\gamma)\big(I - \gamma\,\mathcal{P}_{\pi_\theta}^{\top}\big)^{-1}\rho, \qquad \mathcal{P}_{\pi_\theta}(s, s') = \sum_{a\in\mathcal{A}} \pi_\theta(a \mid s)\,\mathcal{P}(s' \mid s, a), \tag{15}$$

so each $F_k(\theta)$ is an explicit differentiable function of θ and $\nabla F_k(\theta)$ is obtained by automatic differentiation. No trajectories are sampled at any point, and every gradient is exact to machine precision.

Both methods start from $\theta_0 = 0$, receive the same estimate of the smoothness constants L_k, and are given the same total budget of gradient evaluations with the same checkpoint cadence. The uniform baseline (Algorithm 7) has no tolerance parameter, and the worst-case error it attains is governed instead by the grid resolution r; we therefore run it at several resolutions on each task and report the best one it achieves. Each curve is truncated at the last checkpoint at which it still improved, so that no method is represented by a long flat tail. All runs are executed sequentially so that the reported CPU times are not affected by contention.

4.2.1 FishWood

The agent alternates between a fishing state and the woods, and must divide its time between catching fish and collecting wood. The task is a regularized MDP with $\mathcal{S} = \{0 : \text{fishing state}, 1 : \text{woods}\}$ and $\mathcal{A} = \{0 : \text{fish}, 1 : \text{collect wood}\}$. Transitions are deterministic: action 0 leads to the fishing state and action 1 leads to the woods. The reward vector $r(s,a) = [r_1(s,a), r_2(s,a)]$ is stochastic, with $r_1(s,a) = 1$ with probability $p_{\text{fish}} = 0.1$ when $s = 0$, and $r_2(s,a) = 1$ with probability $p_{\text{wood}} = 0.9$ when $s = 1$; the two objectives are therefore the discounted counts of fish and of wood. We take $\gamma = 0.995$, $\tau = 0.5$, and π_{ref} uniform, which gives $|\mathcal{S}||\mathcal{A}| = 4$ decision variables.

Figure 4 reports the worst-case error $\epsilon_{\text{sm-stat}}$ against total gradient evaluations and against CPU time, at a matched budget of $50{,}000$ gradient evaluations. Every uniform grid descends quickly and then arrests at a floor it cannot pass, with the best of them stalling at 9.67×10^{-6}; spending the remaining budget at those same weights produces no further improvement. The adaptive bundle method instead keeps redistributing its effort towards the weights at which the progress criterion is largest, passes below each of those floors in turn, and ends at 4.08×10^{-7} — a factor of 23.7 below the best grid. Figure 4 also includes the equal-spaced-weight baseline of [JHC26], which solves each of its 11 weights to convergence with Adam and reaches 1.54×10^{-4} before it, too, stops improving.

Figure 5 shows the corresponding linear scalarization fronts in reward space and in loss space. All three methods lie on the same front and differ only in how densely they populate it: the two baselines place 11 points each, while the adaptive method returns $1{,}001$ bundle points along the same curve. In reward space the front is a straight line of slope -9, which is a property of the task rather than of any method: FishWood is a pure time-allocation problem, so with fixed success probabilities the rate of exchange between the two objectives is the constant $p_{\text{wood}}/p_{\text{fish}} = 9$. In loss space the same front bends into a convex curve, since F_k adds the τ KL term, which varies from point to point.

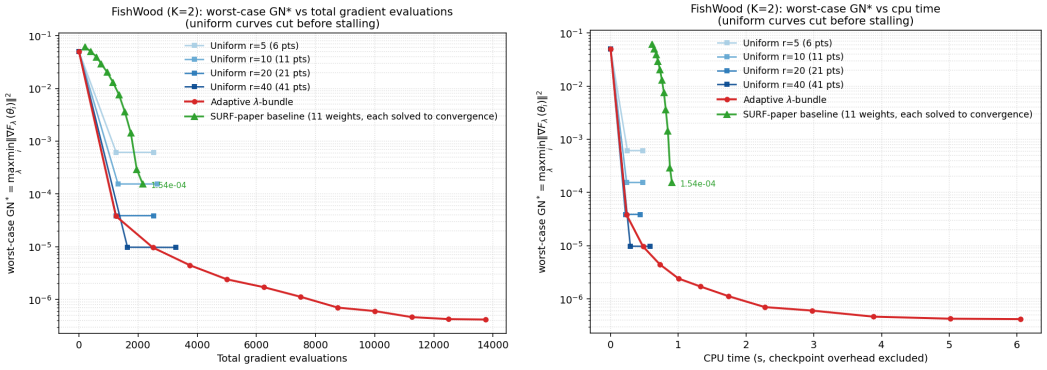

Figure 4: FishWood: worst-case squared gradient norm versus total gradient evaluations (left) and CPU time (right), at a matched budget of $50{,}000$ gradient evaluations. Each uniform grid stalls at a resolution-dependent floor, while the adaptive bundle method continues to descend past all of them.

4.2.2 Deep Sea Treasure (DST)

The agent navigates a grid world containing obstacle cells, treasure cells and water, trading the time spent searching against the value of the treasure it eventually reaches. The state space $\mathcal{S}$ consists of

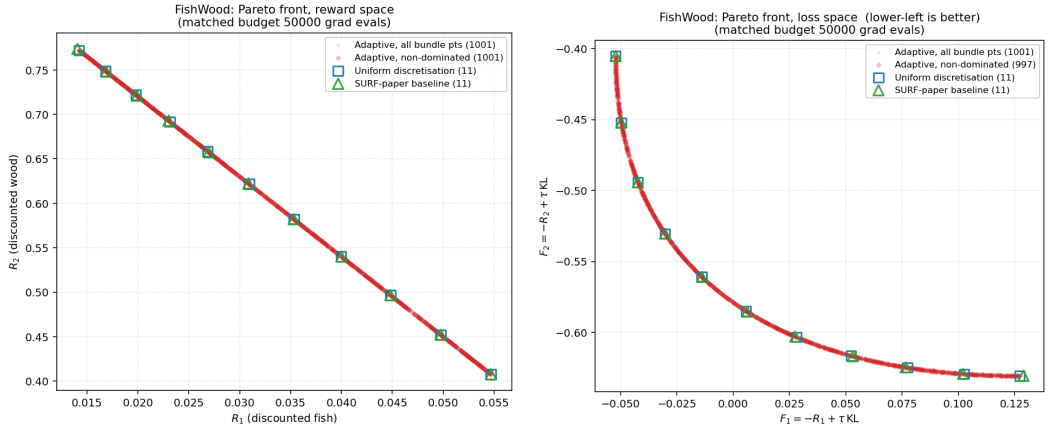

Figure 5: FishWood: linear scalarization front in reward space (left) and in loss space (right). The three methods trace the same front and differ only in how densely they cover it.

the 72 non-obstacle cells together with an absorbing terminal state, and $\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}$. Transitions are deterministic: an action moves the agent to the adjacent cell in the chosen direction, or leaves it in place if that direction is outside the grid or blocked by an obstacle. The reward vector is $r(s,a) = [r_1(s,a), r_2(s,a)]$, where $-r_1(s,a) = 1$ is the time cost incurred at each step until a treasure is reached and $r_2(s,a)$ is the treasure value obtained on reaching a treasure cell, zero otherwise. We take $\gamma = 0.999$, $\tau = 1.5$, and π_{ref} uniform, giving $|\mathcal{S}||\mathcal{A}| = 292$ decision variables. We use the concave variant of the benchmark, in which the treasure values are chosen so that the true Pareto front is nonconvex.

Figure 6 reports the worst-case error at a budget of $200{,}000$ gradient evaluations. As before, each uniform grid flattens out at a floor of its own, the best reaching 5.57×10^{-8}, while the adaptive bundle method continues to descend and ends at 7.17×10^{-9}, a factor of 7.8 lower.

Figure 7 shows the fronts, which exhibit a wide gap in the middle with the solutions clustered near the two ends. This reflects the concave variant of the benchmark: a point in a concave region of the Pareto front, though Pareto stationary, is a saddle rather than a minimum of the corresponding scalarization, so descent on F_λ moves away from it. All the methods compared here descend on scalarized objectives and are affected in the same way. We emphasize that this does not restrict the quantity in (7), which measures stationarity rather than global optimality; the adaptive method still drives it to 7.17×10^{-9} on this task.

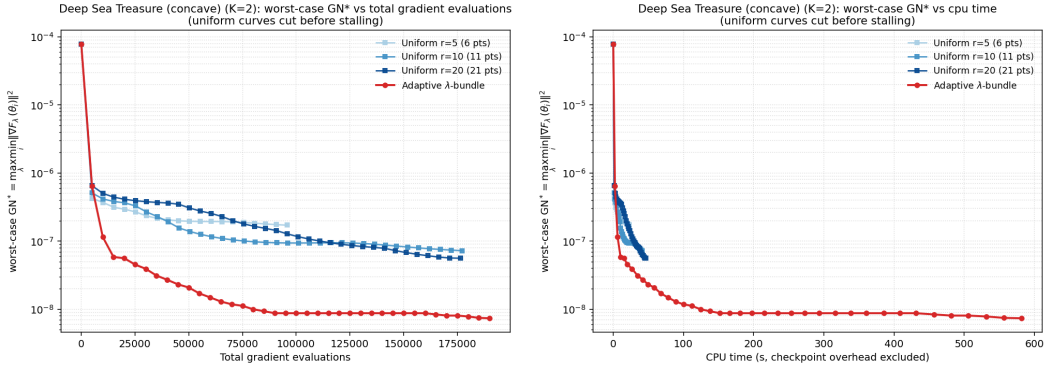

Figure 6: Deep Sea Treasure: worst-case squared gradient norm versus total gradient evaluations (left) and CPU time (right), at a budget of $200{,}000$ gradient evaluations.

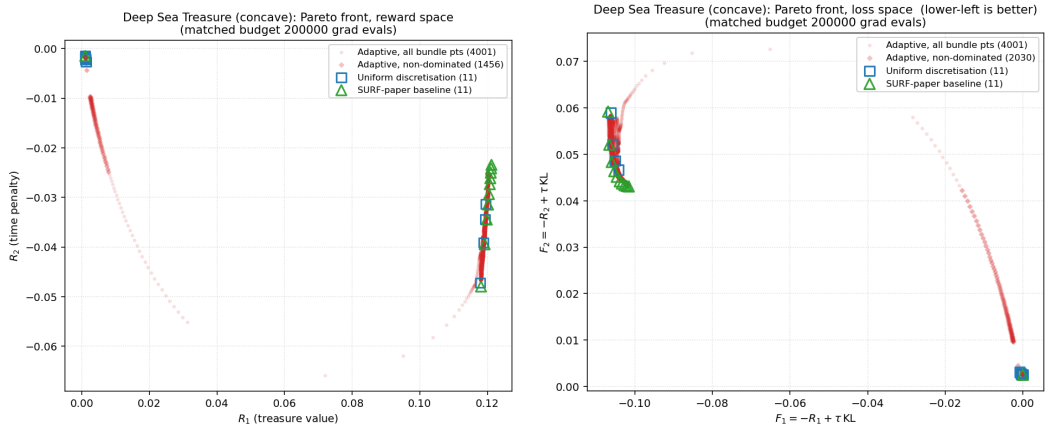

Figure 7: Deep Sea Treasure: linear scalarization front in reward space (left) and in loss space (right). The gap in the middle of the front reflects the nonconvexity of the concave variant's Pareto front and affects all methods equally.

4.2.3 Breakable Bottles

The agent carries bottles from a source to a destination along a corridor of five cells, and may irreversibly break them on the way. The state records the agent's location, the number of bottles it is carrying (0, 1 or 2), the number it has delivered (0, 1 or 2), and a three-bit indicator of which cells hold a dropped bottle; the action set is $\mathcal{A} = \{0 : \text{move left}, 1 : \text{move right}, 2 : \text{pick up a bottle}\}$. Transitions are stochastic: while carrying two bottles, each move drops one with probability 0.1, and a dropped bottle cannot be recovered. The reward vector $r(s,a) = [r_1(s,a), r_2(s,a), r_3(s,a)]$ consists of a time penalty $r_1(s,a) = -1$ at each step, a delivery reward $r_2(s,a)$ for each bottle delivered, and a potential-based term $r_3(s,a)$ penalizing bottles left on the ground. The episode terminates once two bottles have been delivered. We take $\gamma = 0.99$, $\tau = 1.0$, and π_{ref} uniform, which gives 361 states and $|\mathcal{S}||\mathcal{A}| = 1083$ decision variables.

Figure 8 reports the worst-case error at a matched budget of $50{,}000$ gradient evaluations. Covering Δ_K uniformly becomes expensive as K grows, since the number of grid nodes on Δ_3 is $\binom{r+2}{2}$: the resolutions we run require between 15 and 91 separate scalarized problems to be solved out of a single budget, against between 6 and 41 for the corresponding resolutions at $K = 2$. Each grid again stalls at a floor, the best at 1.31×10^{-5}, whereas the adaptive method ends at 8.65×10^{-7}, a factor of 15.2 lower.

Figure 9 shows the non-dominated fronts in three dimensions, in reward space and in loss space, from three viewing angles. The adaptive method recovers 18 non-dominated points against 5 for the uniform grid at the resolution shown, so the improvement in worst-case stationarity is accompanied by a correspondingly richer description of the front itself.

4.2.4 Fruit Tree(To be edit)

Fruit tree problem raises the number of objectives to $K = 6$. The environment is a full binary tree of depth d, in which every leaf holds a fruit with a value for each of six nutrients. The state records the agent's location in the tree and the action set is $\mathcal{A} = \{0 : \text{go left}, 1 : \text{go right}\}$, with deterministic transitions; the agent therefore chooses a root-to-leaf path, and the reward vector $r(s,a) = [r_1(s,a), \ldots, r_6(s,a)]$ collects the six nutrient values of the leaf it reaches. We take $\gamma = 0.99$, $\tau = 1.0$, and π_{ref} uniform, and report both $d = 5$ and $d = 6$, which give $|\mathcal{S}||\mathcal{A}| = 128$ and 256 decision variables respectively.

Figures 11 and 12 report the worst-case error at a matched budget of $50{,}000$ gradient evaluations. At depth 5 the adaptive method ends at 4.30×10^{-6} against 4.95×10^{-6} for the best uniform grid, a factor of 1.15; at depth 6 it ends at 1.74×10^{-6} against 5.91×10^{-6}, a factor of 3.39.

Details: SURF paper: [JHC26]

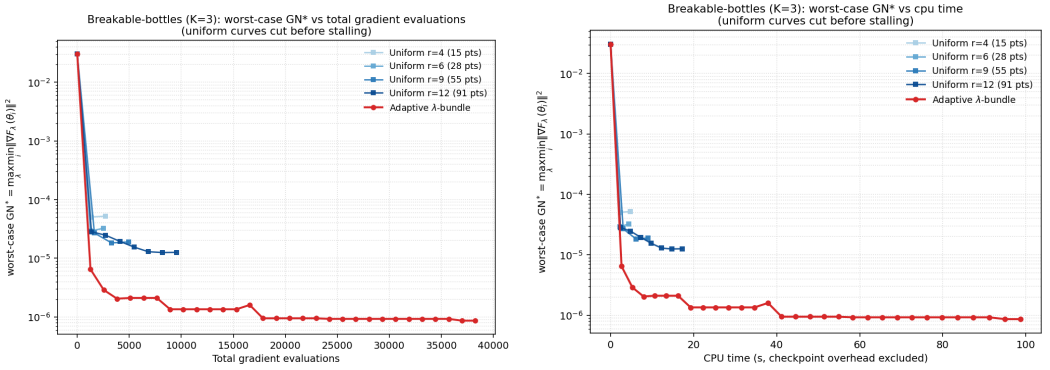

Figure 8: Breakable Bottles ($K = 3$): worst-case squared gradient norm versus total gradient evaluations (left) and CPU time (right), at a matched budget of $50{,}000$ gradient evaluations.

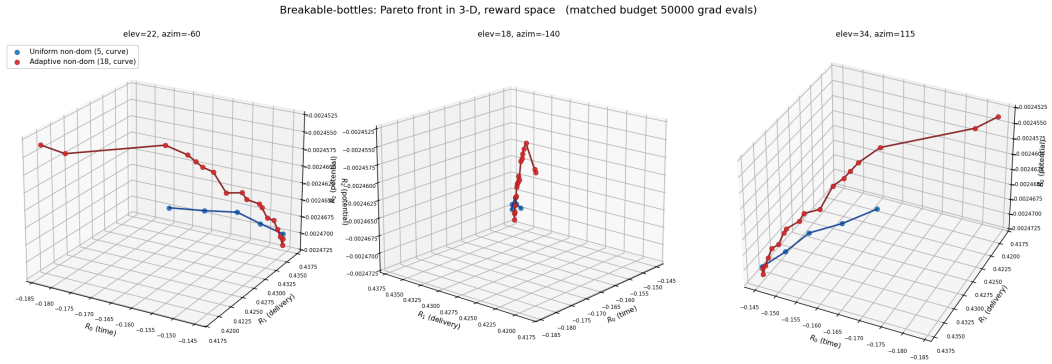

Figure 9: Breakable Bottles: non-dominated front in reward space, shown from three viewing angles. The adaptive method recovers 18 non-dominated points against 5 for the uniform grid.

4.3 DPO Loss Weighting for LLM Alignment

We next evaluate the proposed adaptive bundle method on a two-objective LLM alignment problem. We consider a DPO loss weighting (DPOLW) formulation with two preference objectives: helpfulness and harmlessness. Given a preference dataset $D_k := \{(x^i, y_w^i, y_l^i)\}_{i=1}^{n_k}$ for objective k, the empirical DPO objective is

$$\begin{aligned} F_k(\theta) &:= -\frac{1}{n_k}\sum_{i=1}^{n_k} \log \sigma(z_i(\theta)), \\ z_i(\theta) &:= \beta\left[\log \frac{\pi_\theta(y_w^i \mid x^i)}{\pi_{\text{sft}}(y_w^i \mid x^i)} - \log \frac{\pi_\theta(y_l^i \mid x^i)}{\pi_{\text{sft}}(y_l^i \mid x^i)}\right]. \end{aligned} \tag{16}$$

Because π_θ is a neural language model, each objective F_k is non-convex. For a scalarization weight $\lambda \in [0, 1]$, the two-objective DPOLW problem is

$$\min_\theta F_\lambda(\theta) := \lambda F_{\text{helpful}}(\theta) + (1-\lambda) F_{\text{harmless}}(\theta). \tag{17}$$

Sweeping λ therefore traces an empirical helpfulness–harmlessness trade-off curve.

Experimental protocol. We use `Qwen2.5-0.5B-Instruct` as the base model and fine-tune only LoRA parameters. The preference data are constructed from PKU-SafeRLHF-10K with two objectives, helpfulness and harmlessness. To induce a controlled trade-off, we use a fixed mixture of examples where the helpful and harmless preferences agree and disagree. In the main experiment, the oracle subset contains 64 examples per objective. All methods use the same model, DPO loss, LoRA configuration, oracle subset, and total update budget. We set $\beta = 0.05$, use LoRA rank 8 with

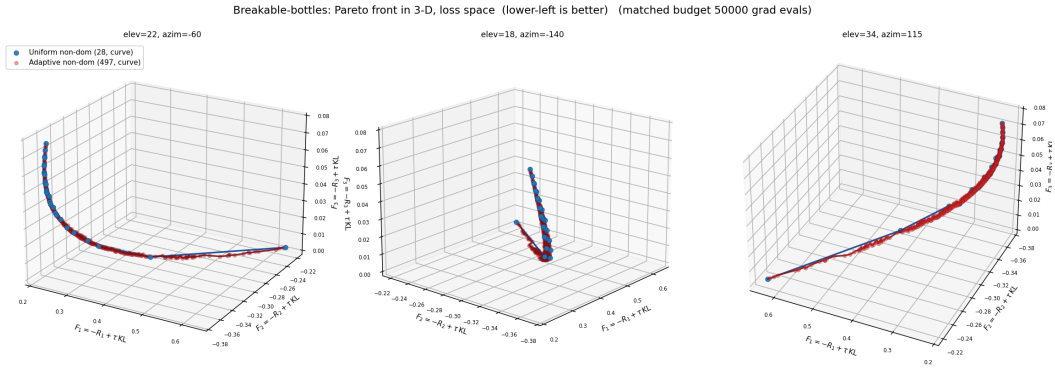

Figure 10: Breakable Bottles: non-dominated front in loss space, shown from three viewing angles.

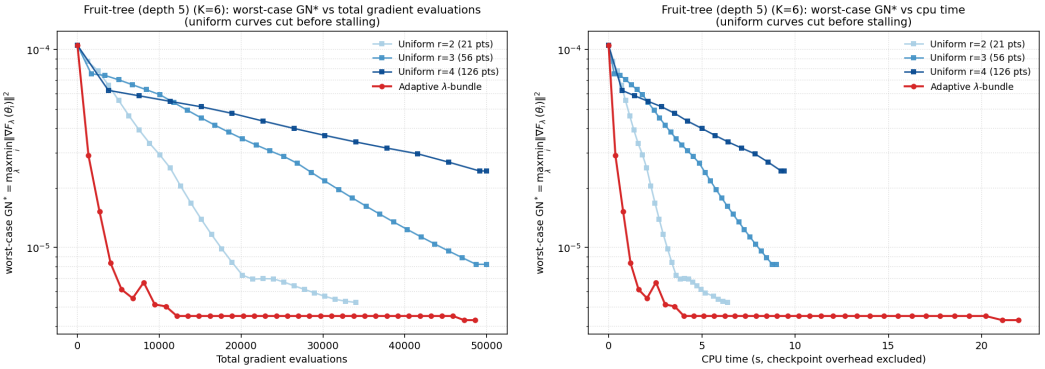

Figure 11: Fruit Tree, depth 5 ($K = 6$): worst-case squared gradient norm versus total gradient evaluations (left) and CPU time (right), at a matched budget of $50{,}000$ gradient evaluations.

LoRA alpha 16, and run each method for a matched budget of 30 outer rounds and 10 inner updates per round, for 300 total parameter updates.

Compared methods. We compare three scalarization strategies. The first is uniform DPOLW, which trains checkpoints on a fixed grid of scalarization weights. The second is our adaptive bundle method, which selects the next weight by maximizing the current worst-case stationarity gap over the bundle. Since this experiment has $K = 2$ objectives, we use the exact one-dimensional lambda solver, which evaluates the lower envelope of quadratic functions using cached Gram matrices instead of relying on a generic nonsmooth nonlinear solver. The third method is SURF [JHC26], which adaptively reallocates scalarization weights using an estimated arc-length distribution of the current Pareto front. SURF is included as a coverage-oriented adaptive scalarization baseline.

Stationarity metric. The primary metric is the best-so-far worst-case gradient norm over the current bundle $\mathcal{B}$:

$$\mathrm{GN}^{\star}(\mathcal{B}) := \max_{\lambda \in [0,1]} \min_{\theta_i \in \mathcal{B}} \|\lambda \nabla F_{\mathrm{helpful}}(\theta_i) + (1-\lambda)\nabla F_{\mathrm{harmless}}(\theta_i)\|_2^2 . \tag{18}$$

A smaller value indicates that the bundle contains, for every scalarization weight, at least one checkpoint that is closer to first-order stationarity. This metric directly matches the goal of the adaptive bundle method, while uniform DPOLW and SURF are evaluated under the same metric for comparison.

Results. Figure 13 reports the best-so-far $\mathrm{GN}^{\star}$ against both objective-gradient evaluations and wall-clock time. The adaptive bundle method reduces the worst-case stationarity gap faster than uniform DPOLW, showing that adaptive weight selection can use the same update budget more

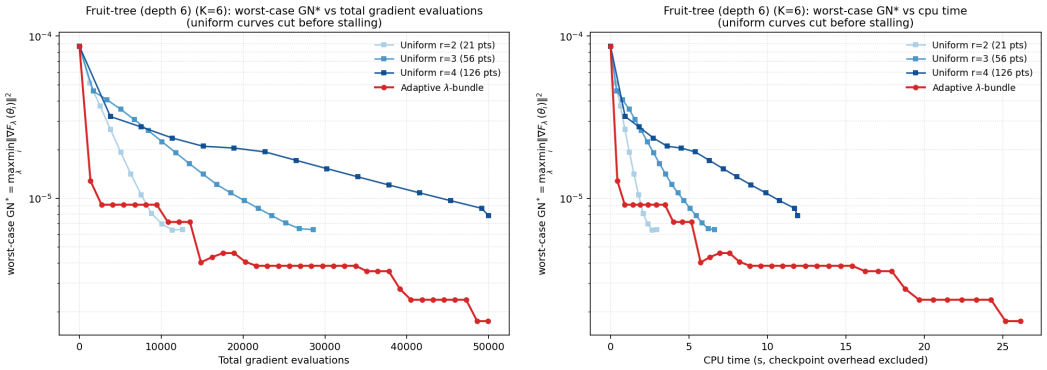

Figure 12: Fruit Tree, depth 6 ($K = 6$): worst-case squared gradient norm versus total gradient evaluations (left) and CPU time (right), at a matched budget of $50{,}000$ gradient evaluations.

efficiently than a fixed grid. SURF also improves over uniform DPOLW, but its progress is slower under the $\mathrm{GN}^\star$ metric because SURF is designed to improve Pareto-front coverage rather than directly minimize the worst-case stationarity gap.

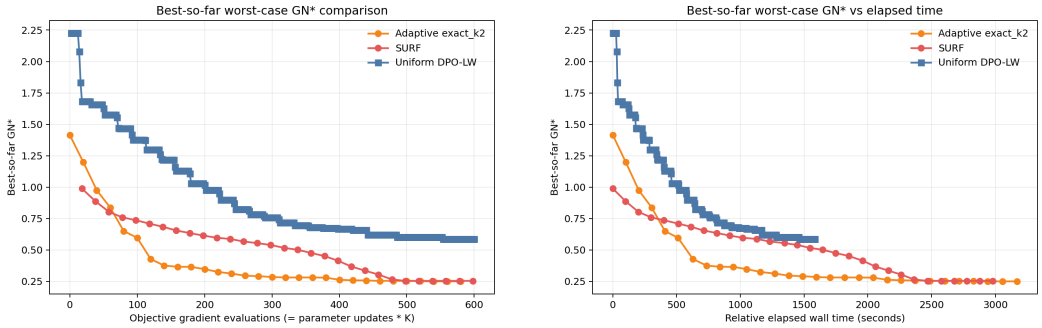

Figure 13: Best-so-far worst-case $\mathrm{GN}^\star$ on the DPOLW alignment task with oracle size 64. Left: objective-gradient evaluations. Right: wall-clock time.

Figure 14 shows the corresponding DPO-loss Pareto fronts. Uniform DPOLW provides a fixed-grid reference front, SURF produces a smooth coverage-oriented front by redistributing scalarization weights, and the adaptive bundle method concentrates computation on weights that are most useful for reducing the current stationarity gap. These results highlight an important distinction: SURF targets uniform traversal of the objective front, whereas our method targets worst-case first-order stationarity over the entire scalarization simplex.

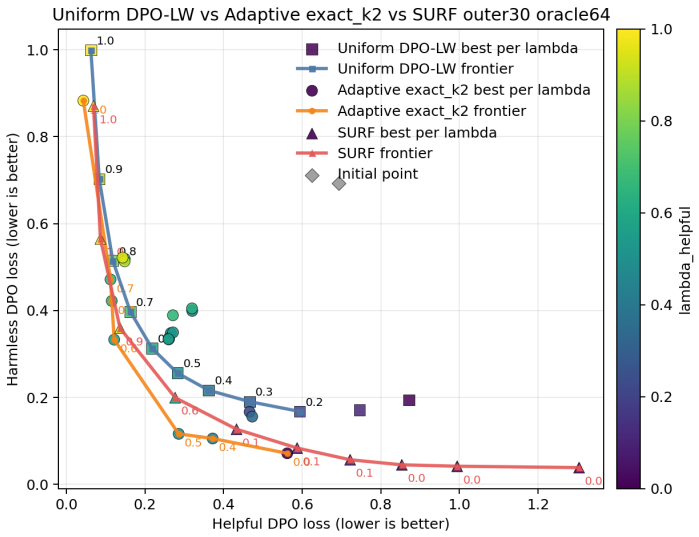

Figure 14: DPO-loss Pareto front for uniform DPOLW, adaptive bundle with exact $K = 2$ lambda selection, and SURF.

Finally, we evaluate the generated checkpoints using external reward and cost models. We report both oracle-seen and oracle-heldout evaluation. The oracle-seen setting evaluates on prompts from the fixed oracle subset used for gradient and objective evaluation, while the oracle-heldout setting uses prompts sampled from the same training pool after removing the oracle examples. This gives a complementary view of whether the DPO-loss front corresponds to improved helpfulness–harmlessness behavior under external reward models. Figure 15 shows the reward-safety Pareto fronts for both settings.

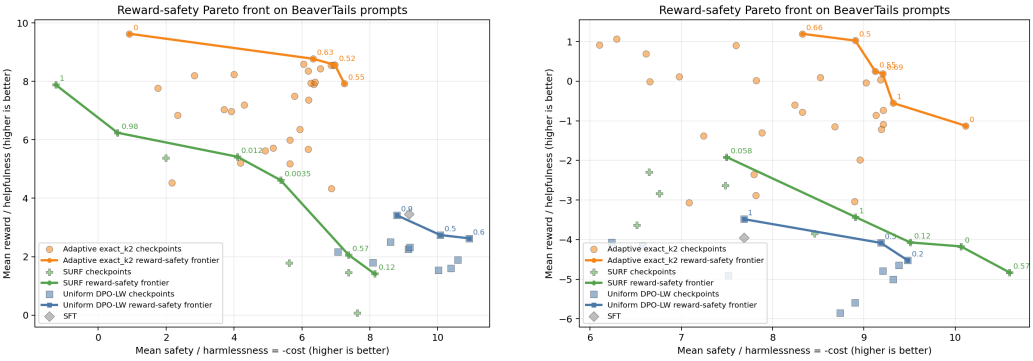

Figure 15: Reward-safety Pareto fronts evaluated by external reward and cost models. Left: oracle-seen prompts. Right: oracle-heldout prompts.

Overall, the LLM experiment demonstrates that the adaptive bundle method can be integrated into a practical multi-objective DPO pipeline with minimal changes to the underlying trainer. The exact $K = 2$ solver removes the need for IPOPT/SLSQP-style lambda optimization in the bi-objective setting, while the bundle mechanism provides a stationarity-driven alternative to both uniform scalarization and SURF-style coverage adaptation.

5 Conclusions and future directions

We proposed a first-order bundle method with convergence guarantee for computing the linear scalarization front of non-convex multi-objective optimization. In contrast to uniform discretization, which solves each scalarized subproblem independently, our method maintains a *bundle* of zeroth and first-order information of the objective functions and reuses it across the simplex. From this bundle it constructs a progress criterion that certifies worst-case stationarity *uniformly* over all weight vectors $\lambda \in \Delta_K$.

We established both iteration and oracle complexity bounds for the resulting algorithm under generic non-convexity (Theorem 1), and we demonstrated empirically that the adaptive method reaches the worst-case stationarity faster than the uniform discretization method at a substantially smaller computation and oracle budget.

Several directions remain open. First, our analysis targets the generic non-convex regime; sharper rates should be available under additional structure such as the strong convexity and Polyak–Łojasiewicz condition, which holds for many over-parameterized learning problems and would tighten the inner-loop query complexity.

Secondly, our adaptive idea could be applied to solving smooth Tchebycheff scalarization for multi-objective optimization, which has applications in tracing the solutions on the non-convex part of the optimal linear scalarization front [LZY+24].

Thirdly, our adaptive bundle method could be extended to the "auto-conditioned/parameter-free" setting where the Lipschitz constant of the gradient is unknown.

Fourthly, our adaptive bundle method could be extended to the constrained setting.

Acknowledgements

References

[Bis06] C.M. Bishop. *Pattern recognition and machine learning*, volume 4. Springer New York, 2006.

[Dés12] Jean-Antoine Désidéri. Multiple-gradient descent algorithm (MGDA) for multiobjective optimization. *Comptes Rendus Mathématique*, 350(5–6):313–318, 2012.

[FAZ+21] Christopher Fifty, Ehsan Amid, Zhe Zhao, Tianhe Yu, Rohan Anil, and Chelsea Finn. Efficiently identifying task groupings for multi-task learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[Geo67] Arthur Geoffrion. Solving bicriterion mathematical programs. *Operations Research*, 15:39–54, 02 1967.

[JHC26] Liuyuan Jiang, Chentong Huang, and Lisha Chen. Surf: Steering the scalarization weight to uniformly traverse the pareto front, 2026.

[Lan20] Guanghui Lan. *First-order and Stochastic Optimization Methods for Machine Learning*. Springer Series in the Data Sciences. Springer, 2020.

[LLJ+21] Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[LZY+24] Xi Lin, Xiaoyuan Zhang, Zhiyuan Yang, Fei Liu, Zhenkun Wang, and Qingfu Zhang. Smooth tchebycheff scalarization for multi-objective optimization. 2024.

[Nes18] Yurii Nesterov. *Lectures on Convex Optimization*. Springer Publishing Company, Incorporated, 2nd edition, 2018.

[NSA+22] Aviv Navon, Aviv Shamsian, Idan Achituve, Haggai Maron, Kenji Kawaguchi, Gal Chechik, and Ethan Fetaya. Multi-task learning as a bargaining game. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16428–16446. PMLR, 2022.

[R97] CARUANA R. Multitask learning : A knowledge-based source of inductive bias. *Machine Learning*, 28:41–75, 1997.

[RHS+16] Sashank J. Reddi, Ahmed Hefny, Suvrit Sra, Barnabas Poczos, and Alex Smola. Stochastic variance reduction for nonconvex optimization, 2016.

[SK18] Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. 31, 2018.

[SZC+20] Trevor Standley, Amir Zamir, Dawn Chen, Leonidas Guibas, Jitendra Malik, and Silvio Savarese. Which tasks should be learned together in multi-task learning? In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[YBHNL26] Hoyeol Yoon, Seoungbin Bae, Nam Ho-Nguyen, and Dabeen Lee. Chebyshev center-based direction selection for multi-objective optimization and training pinns. 2026.

[ZLS+24] Zhanhui Zhou, Jie Liu, Jing Shao, Xiangyu Yue, Chao Yang, Wanli Ouyang, and Yu Qiao. Beyond one-preference-fits-all alignment: Multi-objective direct preference optimization. 2024.

A Omitted Proofs

A.1 Proof of Proposition 1

Proof. $\mathrm{GN}(\lambda;\mathcal{B}') = \min_{\theta_i\in\mathcal{B}'\supseteq\mathcal{B}}\{\|\nabla F_\lambda(\theta_i)\|\} \le \min_{\theta_i\in\mathcal{B}}\{\|\nabla F_\lambda(\theta_i)\|\} = \mathrm{GN}(\lambda;\mathcal{B})$. □

A.2 Proof of Proposition 2

Proof. Define

$$B := \max_{k\in[K]} F_k(\theta_0), \quad b := \min_{k\in[K]} F_k^*.$$

At outer iteration t, the descent lemma applied to $\theta_m^{(t)} = \theta_{m-1}^{(t)} - \frac{1}{L_{\lambda_t}}\nabla F_{\lambda_t}(\theta_{m-1}^{(t)})$ gives

$$F_{\lambda_t}\big(\theta_m^{(t)}\big) \;\le\; F_{\lambda_t}\big(\theta_{m-1}^{(t)}\big) - \frac{1}{2L_{\lambda_t}}\big\|\nabla F_{\lambda_t}\big(\theta_{m-1}^{(t)}\big)\big\|^2 \tag{19}$$

for every $m \ge 1$. Chaining (19) from m down to 0,

$$\sum_{j=1}^{K}\lambda_{t_j}F_j\big(\theta_m^{(t)}\big) = F_{\lambda_t}\big(\theta_m^{(t)}\big) \;\le\; F_{\lambda_t}\big(\theta_0^{(t)}\big) - \frac{1}{2L_{\lambda_t}}\big\|\nabla F_{\lambda_t}\big(\theta_0^{(t)}\big)\big\|^2 \;\le\; F_{\lambda_t}\big(\theta_0\big) - \frac{1}{2L_{\lambda_t}}\big\|\nabla F_{\lambda_t}\big(\theta_0\big)\big\|^2 \le F_{\lambda_t}(\theta_0) \le B.$$

Let $k(t)$ be such that $\lambda_{k(t)} \ge \frac{1}{K}$. Then,

$$B \ge \sum_{j=1}^{K}\lambda_{t_j}F_j\big(\theta_m^{(t)}\big) \ge \lambda_{k(t)}F_{k(t)}(\theta_m^{(t)}) + (1-\lambda_{k(t)})b.$$

Rearranging gives

$$\frac{F_{k(t)}(\theta_m^{(t)}) - b}{K} \le \lambda_{k(t)}[F_{k(t)}(\theta_m^{(t)}) - b] \le B - b \iff F_{k(t)}(\theta_m^{(t)}) \le K(B-b) + b,$$

which is bounded by Assumption 2.2.

Define sublevel set

$$S_k := \{\theta \in \mathbb{R}^d : F_k(\theta) \le K(B-b)+b\}.$$

By the bounded level set assumption, S_k is bounded. Then, $\Theta := \cup_{k=1}^K S_k$ is also bounded, which is exactly Assumption 3.2. □

A.3 Proof of Proposition 3

Proof. Let $\Theta \subseteq \mathbb{R}^d$ be the compact set in Assumption 3.2. One example of LipGN is

$$\mathrm{LipGN} := \max_{\theta\in\Theta}\max_{k\in[K]}\|\nabla F_k(\theta)\|,$$

which is finite by applying the extreme value theorem to compact set Θ and continuous function $f(\theta) := \max_{k\in[K]}\|\nabla F_k(\theta)\|$.

To see this, note that for any bundle $\mathcal{B}$ restricted to the decision space Θ, and for any $\lambda_1, \lambda_2 \in \Delta_K$, we have

$$\begin{aligned}
|\mathrm{GN}(\lambda_1;\mathcal{B}) - \mathrm{GN}(\lambda_2;\mathcal{B})| &= |\min_{\theta\in\mathcal{B}}\{\|\nabla F_{\lambda_1}(\theta)\|\} - \min_{\theta\in\mathcal{B}}\{\|\nabla F_{\lambda_2}(\theta)\|\}| \\
&\le \max_{\theta\in\mathcal{B}}\{|\|\nabla F_{\lambda_1}(\theta)\| - \|\nabla F_{\lambda_2}(\theta)\||\} \\
&\le \max_{\theta\in\mathcal{B}}\{\|\nabla F_{\lambda_1}(\theta) - \nabla F_{\lambda_2}(\theta)\|\} \\
&\le \max_{\theta\in\Theta}\{|J_F(\theta)^\top\lambda_1 - J_F(\theta)^\top\lambda_2|\} \\
&= \max_{\theta\in\Theta}\{|J_F(\theta)^T(\lambda_1-\lambda_2)|\} \\
&\le \max_{\theta\in\Theta}\|J_F(\theta)\|_\infty\|\lambda_1-\lambda_2\|_1 \\
&= [\max_{\theta\in\Theta}\max_{k\in[K]}\|\nabla F_k(\theta)\|]\|\lambda_1-\lambda_2\|_1 \\
&= \mathrm{LipGN}\|\lambda_1-\lambda_2\|_1
\end{aligned}$$

□

A.4 Proof of Lemma 1

Proof. Because the runs are independent, it holds that

$$\mathbb{P}(\min_{\bar{\theta}\in\mathcal{B}} \|\nabla F_\lambda(\bar{\theta})\| = \mathrm{GN}(\lambda;\mathcal{B}) \geq \frac{C}{M^\alpha}) = \prod_{\bar{\theta}\in\mathcal{B}} \mathbb{P}(\|\nabla F_\lambda(\bar{\theta})\| \geq \frac{C}{M^\alpha}) \leq \prod_{s=1}^{S} \frac{1}{2} = \frac{1}{2^S}.$$

Let $\delta = \frac{1}{2^S}$. Solving for $S(\delta)$ gives that

$$S(\delta) = \lceil \log_2 \frac{1}{\delta} \rceil. \tag{20}$$

□

A.5 Proof of Proposition 5

Proof. **GD inner solver.** Nesterov's descent analysis in Section 1.2.3 yields

$$\min_{\theta_i\in\mathcal{B}} \|\nabla F_\lambda(\theta_i)\|^2 \leq \frac{1}{M}\left[\frac{L_\lambda}{\omega}\big(F_\lambda(\theta_0) - F_\lambda^*\big)\right],$$

see [Nes18], eqs. (1.2.22). By picking gradient descent step size $\frac{1}{L_\lambda}$, we have $\omega := \frac{1}{2}$. Hence,

$$\mathrm{GN}(\lambda;\mathcal{B}) = \min_{\theta_i\in\mathcal{B}} \|\nabla F_\lambda(\theta_i)\| = \sqrt{\min_{\theta_i\in\mathcal{B}} \|\nabla F_\lambda(\theta_i)\|^2} \leq \frac{\sqrt{2L_\lambda[F_\lambda(\theta_0) - F_\lambda^*]}}{M^{\frac{1}{2}}}.$$

And we can define $C_\lambda^{GD} := \sqrt{2L_\lambda[F_\lambda(\theta_0) - F_\lambda^*]}$ which is upper bounded by Assumption 2.2.

RSGD inner solver. Apply Corollary 6.1 in [Lan20] to get following inequality:

$$\frac{1}{L_\lambda}\mathbb{E}[\|\nabla F_\lambda(\theta_R)\|^2] \leq \frac{L_\lambda \tilde{D}_\lambda^2}{M} + \frac{2\tilde{D}_\lambda\sigma}{\sqrt{M}} \leq \frac{L_\lambda\tilde{D}_\lambda^2 + 2\tilde{D}_\lambda\sigma}{\sqrt{M}} =: \mathbb{B}_M \tag{21}$$

Note that we set $\tilde{D}_\lambda = D_f$.

By Markov's inequality, we have

$$\mathbb{P}(\|\nabla F_\lambda(\theta_R)\| \geq \frac{C_\lambda^{RSGD}(\delta)}{M^{\frac{1}{4}}}) = \mathbb{P}(\|\nabla F_\lambda(\theta_R)\|^2 \geq \frac{C_\lambda^{RSGD}(\delta)^2}{M^{\frac{1}{2}}}) \leq \frac{\mathbb{E}[\|\nabla F_\lambda(\theta_R)\|^2]}{\frac{C_\lambda^{RSGD}(\delta)^2}{M^{\frac{1}{2}}}} = \frac{L_\lambda(L_\lambda\tilde{D}_\lambda^2 + 2\tilde{D}_\lambda\sigma)}{C_\lambda^{RSGD}(\delta)^2}.$$

Let $\delta = \frac{L_\lambda(L_\lambda\tilde{D}_\lambda^2+2\tilde{D}_\lambda\sigma)}{C_\lambda^{RSGD}(\delta)^2}$. Solving for $C_\lambda^{RSGD}(\delta)$ gives that

$$C_\lambda^{RSGD}(\delta) = \sqrt{\frac{L_\lambda(L_\lambda\tilde{D}_\lambda^2 + 2\tilde{D}_\lambda\sigma)}{\delta}}. \tag{22}$$

2RSGD inner solver. By Markov's inequality and definition of $\mathbb{B}_M$(21), we have

$$\mathbb{P}(\|\nabla F_\lambda(\bar{\theta}_s)\| \geq \frac{C_\lambda^{2RSGD}}{M^{\frac{1}{4}}}) = \mathbb{P}(\|\nabla F_\lambda(\bar{\theta}_s)\|^2 \geq \frac{(C_\lambda^{2RSGD})^2}{M^{\frac{1}{2}}}) \leq \frac{\mathbb{E}[\|\nabla F_\lambda(\bar{\theta}_s)\|^2]}{\frac{(C_\lambda^{2RSGD})^2}{M^{\frac{1}{2}}}} = \frac{L_\lambda\mathbb{B}_M}{\frac{(C_\lambda^{2RSGD})^2}{M^{\frac{1}{2}}}} = \frac{L_\lambda(L_\lambda\tilde{D}_\lambda^2 + 2\tilde{D}_\lambda\sigma)}{(C_\lambda^{2RSGD})^2}.$$

Let $\frac{1}{2} = \frac{L_\lambda(L_\lambda\tilde{D}_\lambda^2+2\tilde{D}_\lambda\sigma)}{(C_\lambda^{2RSGD})^2}$. Solving for C_λ^{2RSGD} gives that

$$C_\lambda^{2RSGD} = \sqrt{2L_\lambda(L_\lambda\tilde{D}_\lambda^2 + 2\tilde{D}_\lambda\sigma)} \tag{23}$$

Apply Lemma 1 to $\alpha = \frac{1}{4}, C = C_\lambda^{2RSGD}$ to get

$$\mathrm{GN}(\lambda;\mathcal{B}) \leq \frac{C}{M^\alpha}, \text{ with probability at least } 1-\delta.$$

SVRG inner solver. Apply Corollary 3 in [RHS+16] to get the universal constants μ_1, ν_1 such that

$$\mathbb{E}[\|\nabla F_\lambda(\theta)\|^2] \leq \frac{L_\lambda n^{\frac{2}{3}}[F_\lambda(\theta_0) - F_\lambda^*]}{QM(n)\nu_1}.$$

By Markov's inequality, we have

$$\mathbb{P}(\|\nabla F_\lambda(\theta)\| \geq \frac{C_\lambda^{SVRG}(n,\delta)}{[QM(n)]^{\frac{1}{2}}}) = \mathbb{P}(\|\nabla F_\lambda(\theta)\|^2 \geq \frac{C_\lambda^{SVRG}(n,\delta)^2}{QM(n)}) \leq \frac{\mathbb{E}[\|\nabla F_\lambda(\theta)\|^2]}{\frac{C_\lambda^{SVRG}(n,\delta)^2}{QM(n)}} = \frac{n^{\frac{2}{3}} L_\lambda (L_\lambda \tilde{D}_\lambda^2)}{2\nu_1 C_\lambda^{SVRG}(n,\delta)^2}.$$

Let $\delta = \frac{n^{\frac{2}{3}} L_\lambda (L_\lambda \tilde{D}_\lambda^2)}{2\nu_1 C_\lambda^{SVRG}(n,\delta)^2}$. Solving for $C_\lambda^{SVRG}(n,\delta)$ gives that

$$C_\lambda^{SVRG}(n,\delta) = \sqrt{\frac{n^{\frac{2}{3}} L_\lambda (L_\lambda \tilde{D}_\lambda^2)}{2\nu_1 \delta}} \tag{24}$$

2SVRG inner solver. By Markov's inequality, we have

$$\mathbb{P}(\|\nabla F_\lambda(\bar{\theta}_s)\| \geq \frac{C_\lambda^{2SVRG}(n)}{[QM(n)]^{\frac{1}{2}}}) = \mathbb{P}(\|\nabla F_\lambda(\bar{\theta}_s)\|^2 \geq \frac{C_\lambda^{2SVRG}(n)^2}{QM(n)}) \leq \frac{\mathbb{E}[\|\nabla F_\lambda(\bar{\theta}_s)\|^2]}{\frac{C_\lambda^{2SVRG}(n)^2}{QM(n)})} = \frac{n^{\frac{2}{3}} L_\lambda (L_\lambda \tilde{D}_\lambda^2)}{2\nu_1 C_\lambda^{2SVRG}(n)^2}.$$

Let $\frac{1}{2} = \frac{n^{\frac{2}{3}} L_\lambda (L_\lambda \tilde{D}_\lambda^2)}{2\nu_1 C_\lambda^{2SVRG}(n)^2}$. Solving for $C_\lambda^{2SVRG}(n)$ gives that

$$C_\lambda^{2SVRG}(n) = \sqrt{\frac{n^{\frac{2}{3}} L_\lambda (L_\lambda \tilde{D}_\lambda^2)}{\nu_1}} \tag{25}$$

Apply Lemma 1 to $\alpha = \frac{1}{2}, C = C_\lambda^{2SVRG}(n), M = QM(n)$ to get

$$\mathrm{GN}(\lambda; \mathcal{B}) \leq \frac{C}{M^\alpha}, \text{ with probability at least } 1 - \delta.$$

□

A.6 Proof of Theorem 1

Proof. We consider a $\frac{\epsilon}{3}$-split of the error tolerance. Under Assumption 3.2, by Proposition 3, consider an $\frac{\epsilon}{3\mathrm{LipGN}}$-net of Δ_K w.r.t. the ℓ_1-norm. That is, a finite set of points $N \subseteq \Delta_K$ such that, for all $\lambda \in \Delta_K$, there exists $\hat{\lambda} \in N$ with

$$\|\lambda - \hat{\lambda}\|_1 \leq \frac{\epsilon}{3\mathrm{LipGN}}.$$

We have for all $t \geq 1$ and for all $\lambda \in \Delta_K$, there exists $\hat{\lambda} \in N$ such that

$$\mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) \leq \mathrm{GN}(\lambda; \mathcal{B}_t) + \frac{\epsilon}{3}; \qquad \mathrm{GN}(\lambda; \mathcal{B}_t) \leq \mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) + \frac{\epsilon}{3}. \tag{26}$$

Therefore, it suffices to show that $\max_{\hat{\lambda} \in N} \mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ at some iteration t.

At any iteration t of Algorithm 1, Assumption 3.1 guarantees that the inner solver and the bundle update rule returns an updated bundle $\mathcal{B}_t$ such that

$$\mathrm{GN}(\lambda_t; \mathcal{B}_t) \leq \frac{\epsilon}{3}, \text{ with probability } 1 - \delta.$$

We call such an event successful if the precision requirement is satisfied.

Now, let $\hat{\lambda}_t \in N$ be the projection of λ_t onto the net N. Then,

$$\mathrm{GN}(\hat{\lambda}_t; \mathcal{B}_t) \leq \mathrm{GN}(\lambda_t; \mathcal{B}_t) + \frac{\epsilon}{3} \leq \frac{2\epsilon}{3}, \text{ with probability } 1 - \delta \tag{27}$$

by combining with the previous inequality.

Thus, at every iteration t of Algorithm 1, we can examine the list of values $\{\mathrm{GN}(\hat{\lambda}; \mathcal{B}_t)$ for $\hat{\lambda} \in N\}$. Since at least one of these values induces a successful event with probability $1-\delta$, by the global monotonicity property (Proposition 1), it remains successful for all subsequent iterations.

In addition, we claim that the preference weights selected at different successful iterations cannot be projected onto the same point in the net N. Suppose this is false for contradiction. Let $t_1 < t_2$ be two outer iterations of Algorithm 1 corresponding to preference weight maximizers λ_{t_1} and λ_{t_2} such that $\hat{\lambda}_{t_1} = \hat{\lambda}_{t_2} = \hat{\lambda}$. Then, by Proposition 1 and the stopping criterion of Algorithm 1,

$$\mathrm{GN}(\lambda_{t_2}; \mathcal{B}_{t_1}) \geq \mathrm{GN}(\lambda_{t_2}; \mathcal{B}_{t_2-1}) > \epsilon$$

Apply inequalities 26 to get

$$\mathrm{GN}(\hat{\lambda}; \mathcal{B}_{t_1}) \geq \mathrm{GN}(\lambda_{t_2}; \mathcal{B}_{t_1}) - \frac{\epsilon}{3} > \frac{2\epsilon}{3}.$$

But this contradicts with inequality 27.

Note that the total number of successes required before all values in the net induce successful events is bounded by $|N| \leq T_C(\epsilon, K) := \left(1 + \frac{C\mathrm{LipGN}}{\epsilon}\right)^{K-1}$, where C is the covering constant for Δ_K in l_1 that is independent of K.

This is a negative binomial/Pascal distribution.

Specifically, let $Y \sim \mathrm{Pascal}(T_C(\epsilon, K), 1-\delta)$. Then, Y stochastically dominates T, i.e: $Y \succeq_{st} T$. Thus, the total number of iterations T required before it is guaranteed that $\max_{\hat{\lambda} \in N} \mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ has expected value $E[T] \leq \frac{T_C(\epsilon,K)}{1-\delta}$.

As mentioned before, $\max_{\hat{\lambda} \in N} \mathrm{GN}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ ensures that

$$\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) = \max_{\lambda \in \Delta_K} \|\nabla F_\lambda(\hat{\theta}(\lambda))\| = \max_{\lambda_t \in \Delta_K} \min_{\theta_i \in \mathcal{B}_{t-1}} \|\nabla F_{\lambda_t}(\theta_i)\| = \mathrm{GN}_t^* \leq \epsilon.$$

This completes the proof. □

A.7 Proof of Proposition 4

Proof. Similar to Proof A.6, the idea is to find a resolution parameter r such that the uniform simplex grid [31] is an $\frac{\epsilon}{3\mathrm{LipGN}}$-net of Δ_K w.r.t. the l_1-norm. By definition of $\mathcal{G}_r$, for all $\lambda \in \Delta_K$, there exists $\hat{\lambda} \in \mathcal{G}_r$ with

$$\|\hat{\lambda} - \lambda\|_1 \leq \frac{K}{r}.$$

Let $\frac{K}{r} = \frac{\epsilon}{3\mathrm{LipGN}}$ and solve for r gives that $r = \lceil \frac{3K\mathrm{LipGN}}{\epsilon} \rceil$. This shows that $\mathcal{G}_r$ is an $\frac{\epsilon}{3\mathrm{LipGN}}$-net of Δ_K w.r.t. the l_1-norm.

In addition, with the gradient descent inner solver and the minimum gradient norm bundle point select rule, Assumption 3.1 is satisfied with $\eta = \frac{2\epsilon}{3}, \delta = 0, \lambda_t = \hat{\lambda} \in \mathcal{G}_r$, and $M(\eta, \delta)$ is the number of iterations required for the gradient descent method to satisfy the gradient norm precision requirement, i.e:

$$\mathrm{GN}(\lambda_t; \mathcal{B}_t) \leq \eta.$$

This implies that for all $\lambda \in \Delta_K$, there exists $\hat{\lambda} \in \mathcal{G}_r$ such that

$$\mathrm{GN}(\lambda; \mathcal{B}_T) \leq \mathrm{GN}(\hat{\lambda}; \mathcal{B}_T) + \frac{\epsilon}{3} = \epsilon.$$

Hence,

$$\epsilon_{\mathrm{sm-stat}}(\hat{\theta}(\cdot)) = \max_{\lambda \in \Delta_K} \|\nabla F_\lambda(\hat{\theta}(\lambda))\| = \max_{\lambda \in \Delta_K} \min_{\theta \in \mathcal{B}_T} \|\nabla F_\lambda(\theta)\| = \max_{\lambda \in \Delta_K} \mathrm{GN}(\lambda; \mathcal{B}_T) \leq \epsilon.$$

It remains to compute the cardinality of $\mathcal{G}_r$. By stars-and-bars argument, $|\mathcal{G}_r| = \binom{r+K-1}{K-1}$, so the grid size grows exponentially in the number of objectives K. Substituting $r = \lceil \frac{3K\mathrm{LipGN}}{\epsilon} \rceil$ into $\binom{r+K-1}{K-1}$

and using $\lceil x \rceil \leq x + 1$,

$$r + K - 1 \;\leq\; \frac{3K\mathrm{LipGN}}{\epsilon} + K \;=\; K\left(1 + \frac{3\mathrm{LipGN}}{\epsilon}\right). \tag{28}$$

Hence, by $\binom{n}{k} \leq \frac{n^k}{k!}$ applied with $n = r + K - 1$, $k = K - 1$,

$$\binom{r+K-1}{K-1} \;\leq\; \frac{(r+K-1)^{K-1}}{(K-1)!} \;\leq\; \frac{K^{K-1}}{(K-1)!}\left(1 + \frac{3\mathrm{LipGN}}{\epsilon}\right)^{K-1}, \tag{29}$$

where the second inequality is (28).

Using $(K-1)! \geq \left(\frac{K-1}{e}\right)^{K-1}$ and $\left(1 + \frac{1}{K-1}\right)^{K-1} \leq e$, this simplifies to

$$\binom{r+K-1}{K-1} \;\leq\; e^K\left(1 + \frac{3\mathrm{LipGN}}{\epsilon}\right)^{K-1} \tag{30}$$

□

B Omitted Descriptions

B.1 Uniform Discretization — The Baseline Method

In this section, we describe the uniform discretization method, which serves as the baseline method for solving problem 2. The idea is to partition the unit simplex Δ_K into a uniform grid, run a deterministic inner solver at each grid point, and construct a bundle from the solution trajectories to build a solution map.

Let $r \in \mathbb{Z}_+$ be a grid resolution parameter. Define the uniform simplex grid

$$\mathcal{G}_r \;:=\; \big\{\lambda \in \Delta_K \;:\; r\lambda \in \mathbb{Z}_+^K\big\}. \tag{31}$$

That is, $\mathcal{G}_r$ consists of all grid points whose coordinates are multiples of $1/r$.

Algorithm 7 Uniform discretization algorithm

1: **Input:** Resolution r; number of classes K; initial point θ_0; error tolerance ϵ; bundle point select rule $\mathcal{R}$.
2: **Initialize:** Initial bundle $\mathcal{B}_0$ built at θ_0. Enumerate $\mathcal{G}_r = \{\lambda_1, \ldots, \lambda_T\}$ in lexicographic order.
3: **for** iteration $t = 1, \ldots, T$ **do**
4: CandidatePoints = InnerSolver (λ_t)
5: $\mathcal{B}_t = \mathcal{B}_{t-1} \cup \mathcal{R}$(CandidatePoints)
6: **end for**
7: **return** $\mathcal{B}_T$ to build solution map $\hat{\theta}(\lambda) := \arg\min_{\theta_i \in \mathcal{B}_T} \|\nabla F_\lambda(\theta_i)\|$.

lexiographic ordering. We enumerate $\mathcal{G}_r = \{\lambda_1, \ldots, \lambda_T\}$ in lexicographic order, which makes consecutive grid points ℓ_1-close ($\|\lambda_{i+1} - \lambda_i\|_1 \le \frac{2}{r}$).

Solution map construction. To build solution map $\hat{\theta}(\lambda) := \arg\min_{\theta_i \in \mathcal{B}_t} \|\nabla F_\lambda(\theta_i)\|$, the least-index rule is used to break ties.

Inner Solver. We use a deterministic first-order method, e.g: gradient descent (GD) variants, as the inner solver to generate a sequence of points to be added to the current bundle.

Bundle point select rule. After the inner solver generates a list of candidate points, we may update the current bundle by selecting a subset of them. There are many selection rules, e.g: adding only the last point, adding all points, adding a random point, or adding the point with the minimum gradient norm. There are trade-offs among these selection rules. For example, adding all points could improve solution accuracy at the cost of extra oracle calls.

Convergence analysis. The convergence result of Algorithm 7 is presented in Proposition 4.

B.2 Preference Weight Selection

B.2.1 The Criterion

Define, for $i \in [m]$,

$$\phi_i(\lambda) := \|\nabla F_\lambda(\theta_i)\|^2, \quad \mathcal{A}(\lambda) := \arg\min_{i\in[m]} \phi_i(\lambda), \quad \phi(\lambda) := \min_{i\in[m]} \phi_i(\lambda) = \min_{\theta_i\in\mathcal{B}_m} \lambda^\top Q_i \lambda, \quad \text{GNS} := \max_{\lambda\in\Delta_K} \phi(\lambda).$$

At each outer iteration t, the selection step solves

$$\text{GNS} = \max_{\lambda\in\Delta_K} \min_{\theta_i\in\mathcal{B}_m} \lambda^\top Q_i \lambda = \max_{\lambda\in\Delta_K} \{t : t \le \lambda^T Q_i \lambda,\ \forall i \in [m]\}, \tag{32}$$

B.2.2 Curvature

ϕ is a pointwise minimum of convex functions and is in general neither convex nor concave, and GNS is the maximization of a nonconcave function over a polytope. Local maxima of ϕ sit at "crossings" of the lower envelope, not in the interiors of single pieces.

Lemma 2 (Interior maximizers are crossings)**.** *Let λ^* be a local maximizer of ϕ over Δ_K, let S be the smallest face of Δ_K containing λ^*, and let T_S denote the tangent space of S. If $\mathcal{A}(\lambda^*) = \{i\}$, then $J_F(\theta_i)^T v = 0$ for every $v \in T_S$; equivalently, ϕ_i is constant on S.*

Corollary B.1 (Debugging tips)**.** *Except in the degenerate case where some ϕ_i is constant on a face, every local maximizer lying in the relative interior of a face of dimension bigger than or eqaul to 1, in particular, every interior maximizer has at least two active bundle points. The only generic single-active-point maximizers are the vertices $e_1, \dots, e_K$.*

Proposition 6 (Characterization of zero GNS)**.** *GNS=0 if and only if some bundle point is simultaneously stationary for all objectives, i.e: there exists i such that $\nabla F_k(\theta_i) = 0$ for every $k \in [K]$.*

B.2.3 Elementary bounds

Proposition 7 (Free sandwich)**.** *Define zero-sum game $A \in \mathbb{R}^{m\times K}$ by $A_{ik} := \|\nabla F_k(\theta_i)\|^2 = [Q_i]_{kk}$. Let the value of the zero-sum matrix game with payoff A by*

$$val(A) := \max_{\lambda\in\Delta_K} \min_{i\in[m]} (A\lambda)_i = \min_{w\in\Delta_m} \max_{k\in[K]} (A^T w)_k$$

in which the row (bundle) player minimizes. This is a LP in $K+1$ variables. The free sandwich theorem says

$$\max_{k\in[K]} \min_{i\in[m]} A_{ik} \le \textit{GNS} \le val(A) \le \min_{i\in[m]} \max_{k\in[K]} A_{ik}$$

Note that the maximizer λ_A of the middle LP is a good warm start for the ascent method of the CCP algorithm, being the maximizer of a global overestimate of ϕ.

Lemma 3 (Convexification of rank-1 SDP collapses to $val(A)$)**.** *Define the convex hull of rank-1 simplex SDP by*

$$\mathcal{C} := conv(\{\lambda\lambda^T : \lambda \in \Delta_K\}) = \{\Lambda \in \mathcal{CP}_K : \langle 11^T, \Lambda\rangle = 1\}$$

Then,

$$\max_{\Lambda\in\mathcal{C}} \min_{i\in[m]} \langle Q_i, \Lambda\rangle = val(A)$$

Corollary B.2. *Any relaxation obtained by replacing $\mathcal{C}$ with a tractable superset - the Shor relaxation, the DNN cone, or any outer approxiamtion of $\mathcal{CP}_K$ - yields a bound no smaller than $val(A)$, hence no better than the LP of Proposition 7.*

B.2.4 Convex-concave procedure (CCP)

Definition B.1 (Minorant)**.** *A concave piecewise-affine minorant of ϕ at iterate $\lambda_c \in \Delta_K$ is*

$$\psi_c := \min_{i\in[m]} l_i^{(c)}(\lambda) = \min_{i\in[m]} [\phi_i(\lambda_c) + \langle \nabla\phi_i(\lambda_c), \lambda - \lambda_c\rangle] = \min_{i\in[m]} [2\lambda_c^T Q_i \lambda - \lambda_c^T Q_i \lambda_c]$$

which is the tangent plane at λ_c serving as the lower bound of $\phi_i(\lambda)$.

The CCP step maximizes the minorant

$$\lambda_+ \in \arg\max_{\lambda\in\Delta_K} \psi_c(\lambda) = \arg\max_{\lambda\in\Delta_K} \min_{i\in[m]} (M^c\lambda)_i, \text{where } M^c_{ik} := 2\langle \nabla F_k(\theta_i), \nabla F_{\lambda_c}(\theta_i)\rangle - \|\nabla F_{\lambda_c}(\theta_i)\|^2. \tag{33}$$

which can be formulated as a LP:

$$\max t \text{ s.t. } t\vec{1} \le M^c\lambda; 1^T\lambda = 1; \lambda \ge 0$$

This LP can be solved exactly due to concavity. In addition, the LP optimum is a vertex of the epigraph polytope in (t, λ) space, so iterates move freely into the relative interior.

Lemma 4 (Monotone ascent)**.** *Any maximizer λ_+ satisfies $\phi(\lambda_+) \ge \phi(\lambda_c)$.*

Definition B.2 (Stationary iterate)**.** *λ_c is stationary for $\max_{\lambda\in\Delta_K} \phi(\lambda)$ if $\phi'(\lambda_c; v) \le 0$ for every feasible direction $v \in T_{\Delta_K}(\lambda_c)$ where $\phi'(\lambda_c; v) := \min_{v\in\mathcal{A}(\lambda_c)} \langle \nabla\phi_i(\lambda_c), v\rangle$ is the directional derivative of a minimum of smooth functions.*

Lemma 5 (Minorant maximizers are stationary points)**.**

Proposition 8 (Convergence of minorant maximizers)**.** *Let $\{\lambda_c\}_{c\ge0} \subseteq \Delta_K$ be a sequence of minorant maximizers defined at iterates in the unit simplex. Then, $\{\phi(\lambda_c)\}_{c\ge0}$ is non-decreasing and convergent, and every accumulation point of $\{\lambda_c\}$ is stationary for $\max_{\lambda\in\Delta_K} \phi(\lambda)$.*

Algorithm 8 Multi-start CCP

Input: Gram matrices $Q_1, \dots, Q_m \in \mathbb{S}^K_+$; seed count N; restart count r; tolerance τ; iter cap T.
Form zero-sum game $A_{ik} = [Q_i]_{kk}$ and compute the sandwich in Proposition 7
if $\max_{k\in[K]} \min_{i\in[m]} A_{ik} = val(A)$ **then**
  return the corresponding vertex (sandwich closed)
else
  Build a seed set Σ consisting of K vertices, the game maximizer λ_A, and N points drawn uniformly from Δ_K. Evaluate ϕ on all of Σ in one batched contraction; retrain the r best, well-separated seeds.
  for each retained seed λ^0 **do**
    for iteration $j = 0, 1, \dots, T$ **do**
      if $\delta_c \le \tau$ **then**
        Record the limit point and its value and exit the loop
      else
        Form M^c via payoff matrix formula 33
        Solve the LP $\max\{t : t\vec{1} \le M^c\lambda, \lambda \in \Delta_K\}$, warm-start from the previous basis
        Let λ_{j+1} be the LP maximizer and $\delta_c := t^* - \phi(\lambda_c)$
      end if
    end for
  end for
  return the recorded point of largest value (optionally the whole pool of distinct local maximizers)
end if

Definition B.3 (Stopping criterion)**.** *The predicted improvement $\delta_c := val(M^c) - \phi(\lambda_c) \ge 0$.*

- *Stop when $\delta_c \le \tau := 10^{-8}\max\{1, \phi(\lambda_c)\}$.*

Initialization, Multistart, and Reuse.
Seeds. Use the K vertices $\{e_k : k \in [K]\}$; the maximizer λ_A of the zero-sum game; a batch of uniform draws on Δ_K.
Screening then polishing. Evaluate ϕ on all seeds in one batched contraction against the tensor $(Q_i)_{i=1}^m$ and polish only the top handful with CCP. This is cheaper than running CCP from every seed and loses little.
Keep the pool, not just the winner. Distinct local maximizers correspond to genuinely different trade-off regions that the current bundle handles poorly. Retaining several gives several candidate directions per outer round, which parallelizes downstream point generation.
Warm start across outer rounds. Adding one bundle point only dents the lower envelope locally, so the previous round's pool of local maximizers is an excellent seed set for the next round. Carry the pool forward and re-screen it alongside a smaller batch of new samples.

C Additional Figures